

APPLIED PROJECT REPORT

IMPERIAL COLLEGE LONDON
IMPERIAL COLLEGE BUSINESS SCHOOL
MSc RISK MANAGEMENT & FINANCIAL ENGINEERING

Statistical Arbitrage of Equity Index Volatility:
A Machine Learning Approach to Dispersion Trading

Nathan SEBBAG
CID: 06035175

July 30, 2026

The full implementation (code, notebooks, figures and tables) is available at <https://github.com/NSB22/Dispersion-Trading-Project>; the source code is listed in Appendix B.

Contents

1	Introduction	1
2	Theory	2
2.1	Implied correlation	2
2.2	The premium as compensation for correlation risk	2
2.3	The vega-neutral book	2
2.4	Cleaning the correlation matrix	3
3	Data and methodology	4
3.1	Data sources	4
3.2	Universe construction	4
3.3	Realised measures and the two signal objects	4
3.4	The backtest engine	5
3.5	The machine-learning protocol	5
4	Results	6
4.1	The correlation risk premium exists, and has compressed	6
4.2	The baseline trade: real, and nearly killed by frictions	6
4.3	RMT helps, but the value is the estimation window, not the clipping	8
4.4	The ML timing layer is a null; a regime gate trims the tail	9
5	Limitations	12
6	Conclusion	13
	References	14
A	Repository guide	16
A.1	Structure	16
A.2	Setup	16
A.3	Running the pipeline	17
B	Source code	18
B.1	Pipeline entry point	18
B.2	Data acquisition (<code>src/dispersion/data/</code>)	20
B.3	Signal construction (<code>src/dispersion/signal/</code>)	34
B.4	Random matrix theory cleaning (<code>src/dispersion/rmt/</code>)	37
B.5	Backtest engine (<code>src/dispersion/backtest/</code>)	42
B.6	Machine learning layer (<code>src/dispersion/ml/</code>)	52
B.7	Shared utilities (<code>src/dispersion/utils/</code>)	64
B.8	Tests (<code>tests/</code>)	67

1 Introduction

Investors buy index options as portfolio insurance, and that demand shows up in prices. Because index variance is a function of constituent variances and the correlations between them, an expensive index option, for given single-name option prices, mechanically implies an expensive correlation. [Driessen, Maenhout and Vilkov \(2009\)](#), hereafter DMV, showed that the correlation implied by S&P 500 option prices systematically exceeds the correlation that subsequently realises, and that this gap, the correlation risk premium (CRP), is priced: index variance risk carries a large negative premium while individual variance risk carries almost none. The gap is visible in raw data. On 31 March 2020, at the depth of the COVID sell-off, the mean 91-day implied volatility of our top-100 constituents was 46.3% against 37.6% for the index itself. Diversification fills in the difference, and correlation governs how much.

A dispersion trade harvests this premium directly. Sell the index straddle, buy a basket of constituent straddles, and size the legs so the book is vega-neutral: a parallel move in all volatilities then nets to zero, and what remains is a short position in correlation. If realised correlation comes in below the level the market charged, the trade earns the premium.

The brief for this project was to establish whether the CRP still exists on a modern sample, whether a systematic dispersion book can capture it net of realistic costs, and whether two additional layers add value: a Random Matrix Theory (RMT) cleaning of the realised correlation matrix, and a machine-learning timing signal. We measure the premium over 1996 to 2024 (29 years, 7,221 trading days) on a point-in-time top-100 S&P 500 universe built from CRSP and OptionMetrics, backtest the DMV vega-neutral trade under a calibrated cost model, then test the RMT and ML layers under a purged walk-forward protocol.

The results are mixed in an instructive way. The premium exists, at +0.079 on average ($t = 7.4$), but it compressed to a statistically insignificant +0.028 after 2020, which fits a limits-to-arbitrage account. The unconditional trade replicates DMV almost exactly (gross Sharpe 0.77 against their 0.73) and, like theirs, is nearly killed by frictions. The RMT gate transforms the tail profile, though an ablation places the value in the 252-day estimation window rather than the eigenvalue clipping. The supervised ML timing layer is a clean null. We report these nulls plainly; they are findings, not failures.

2 Theory

2.1 Implied correlation

Under a one-factor approximation, in which a single average correlation $\bar{\rho}$ stands in for all pairs, index variance decomposes as

$$\sigma_I^2 = \sum_{i=1}^N w_i^2 \sigma_i^2 + \bar{\rho} \sum_{i \neq j} w_i w_j \sigma_i \sigma_j. \quad (1)$$

Solving for $\bar{\rho}$ and feeding implied volatilities gives the implied correlation:

$$\rho_{\text{implied}} = \frac{\sigma_I^2 - \sum_i w_i^2 \sigma_i^2}{\left(\sum_i w_i \sigma_i\right)^2 - \sum_i w_i^2 \sigma_i^2}, \quad (2)$$

where σ_I is the index implied volatility, σ_i the constituent implied volatilities and w_i the index weights. The denominator is strictly positive for $N \geq 2$, and the formula respects its natural bounds: $\rho_{\text{implied}} \leq 1$ exactly when the index vol sits below the weighted-average constituent vol (the diversification inequality), and $\rho_{\text{implied}} \geq 0$ whenever σ_I^2 exceeds the concentration term, which is tiny at $N = 100$. DMV build the same object from model-free implied variance (MFIV), the Carr and Madan (1998) integral of option prices across strikes, which captures the whole smile rather than a single point. We use at-the-money volatilities instead, a choice we defend in Section 3 and quantify in Section 4.

2.2 The premium as compensation for correlation risk

DMV's identification runs through the variance risk premium (VRP). Applying Itô's lemma to the index decomposition and changing measure, the index VRP splits into a weighted sum of individual VRPs plus terms in the difference between risk-neutral and physical correlation; the convexity corrections cancel in the $\mathbb{Q} - \mathbb{P}$ difference. Empirically, individual variance premia are close to zero (Bakshi & Kapadia, 2003; Bakshi et al., 2003), so the large negative index VRP must be carried by the correlation terms. Selling index variance and buying individual variance isolates precisely that exposure.

2.3 The vega-neutral book

The trade is built from 91-day at-the-money straddles: short the index straddle, long the constituent straddles. The hedge is triangular. First the vega leg: sizes are set so that a proportional shock to all volatilities leaves the book flat. Since an ATM straddle's relative vega is approximately $1/\sigma$, this convention pins the component leg near 100% of the short index notional, which is exactly DMV's +101.12%. The vol-of-vol cancels in the relative-vega ratio, so the hedge needs no vol-dynamics model. Second the delta leg: under the same one-factor structure, the residual delta of the vega-hedged book loads on the index alone while idiosyncratic deltas diversify away, so we hedge daily with a single

index instrument. Daily delta-hedging a straddle is gamma scalping; to first order the hedge P&L accrues at

$$d\Pi \approx \frac{1}{2} \Gamma S^2 (\sigma_{\text{realised}}^2 - \sigma_{\text{implied}}^2) dt, \quad (3)$$

so the hedged book pays off on realised-versus-implied variance, and through the index leg on realised-versus-implied correlation. That is the bet, cleanly stated.

2.4 Cleaning the correlation matrix

The one-factor inversion collapses a 100×100 matrix into one number, and the sample matrix it summarises is noisy. For N assets on T observations with $q = N/T$, a purely random matrix has eigenvalues confined to the [Marchenko and Pastur \(1967\)](#) bulk $[(1 - \sqrt{q})^2, (1 + \sqrt{q})^2]$. Financial spectra show a three-tier structure instead: one large market eigenvalue, a handful of sector modes, and a bulk statistically indistinguishable from noise ([Laloux et al., 1999](#)). Because the market mode absorbs a fraction λ_1/N of the trace, the noise band for the remainder shrinks; [Laloux et al.](#)'s corrected edge is $\lambda_+^{\text{eff}} = (1 - \lambda_1/N)(1 + \sqrt{q})^2$. Our cleaning pipeline diagonalises the de-volatilised matrix, clips bulk eigenvalues to a constant preserving the trace, and renormalises ([Bun et al., 2017](#); [Potters & Bouchaud, 2020](#)). Whether this helps a dispersion trade is an empirical question, and Section 4 answers it more honestly than we expected.

3 Data and methodology

3.1 Data sources

All data come from WRDS. Implied volatilities are taken from the OptionMetrics IvyDB US standardised surface (OptionMetrics, 2025): the 91-day maturity, native to the surface grid, with the $+50\Delta$ call and -50Δ put averaged into an ATM-forward volatility; both legs are required, otherwise the observation is dropped. The index is SPX (`secid` 108105). Entry straddle prices come from the vendor’s `impl_premium` field, which we verified to be forward-consistent: the dividend yield backed out of the two SPX legs is identical (1.04%), while a naive Black–Scholes reprice with $q = 0$ misses by $\pm 5\%$. Returns are CRSP total returns from the daily stock file, index membership is the point-in-time `dsp500list`, and zero rates come from `zerocd` (CRSP, 2025). OptionMetrics and CRSP identifiers are linked through the WRDS bridge, keeping exact (score-1) matches only; an ambiguous link is a silent source of bias, so we refuse it.

3.2 Universe construction

At each quarter-end we rank that day’s actual S&P 500 members by market capitalisation and keep the top 100, with weights renormalised over the basket. $N = 100$ is a deliberate choice. It covers 63–70% of index capitalisation (against roughly 40% at $N = 30$), so the truncation error in the variance identity of eq. (1) stays small; it gives the RMT layer a well-populated spectrum (100 eigenvalues, 4,950 pairs); and coverage is excellent, with 97–100 of 100 names carrying a valid IV in every year since 1996. The sample runs 1996 to 2024, 116 quarterly rebalances, bounded below by the start of the vol surface and above by the end of our CRSP subscription. Turnover is 3–8 names per quarter. We use full market capitalisation rather than free float (the float-weight file sits behind an access wall); the difference is small for mega-caps and documented.

3.3 Realised measures and the two signal objects

Realised volatilities are rolling 63-day standard deviations of log returns, annualised by $\sqrt{252}$ to sit on the same footing as implied volatilities. The realised correlation matrix is estimated complete-case, which keeps it positive semi-definite, and collapsed to a weighted average using the same $w_i\sigma_i$ weights as eq. (2), so the implied and realised averages are the same kind of object. Timing matters here. The IV at date t prices the window $[t, t + 63]$ trading days], while a trailing correlation looks backwards, so we build two series: the tradeable signal $S_t = \rho_{\text{implied},t} - \rho_{\text{realised},t}^{\text{trail}}$, which uses only information available at t , and the ex-post premium $\Pi_t = \rho_{\text{implied},t} - \rho_{\text{realised},t}^{\text{fwd}}$, measured over exactly the window the option priced, obtained by shifting the trailing series back 63 trading days. The premium establishes that the CRP exists; the signal, its ex-ante estimator through correlation persistence, is what the strategy trades. The forward series is never a trading input.

3.4 The backtest engine

At each rebalance the book is set vega-neutral to proportional volatility shocks and anchored at -100% of wealth in the index straddle, DMV’s normalisation, which makes our returns directly comparable to their Table II. Positions are then frozen for the quarter: the vega drifts, and that drift is the accepted convexity risk. Delta is hedged daily with the index alone, which is the hedge implied by the one-factor model itself and costs 1 bp of traded notional instead of a hundred single-name spread crossings. The book is marked to the surface daily (interpolation linear in total variance across maturity pillars) and settles at intrinsic value at the next rebalance, DMV’s held-to-maturity convention. Costs follow their “spread paid once” rule: each entry leg pays half the relative bid–ask spread, calibrated on real quoted spreads from `opprcd` and interpolated linearly between 1996 and 2024 anchors (SPX $1.0\% \rightarrow 0.6\%$; large caps $7.0\% \rightarrow 1.2\%$; small lines $10.0\% \rightarrow 3.0\%$). Quoted spreads overstate effective ones, so net results are a conservative lower bound.

3.5 The machine-learning protocol

The timing layer is a meta-labelling veto (López de Prado, 2018): the RMT-gated strategy proposes each trade and the model may veto it. Features come in three nested sets, which form the central test of the project: A, VIX-family only (level, term slope, VIX regime); B, adding the RMT spectral features (λ_1/N , the absorption ratio of Kritzman et al. (2011), the count K of eigenvalues above the Laloux edge, $\Delta\lambda_1$, dominant-eigenvector rotation, and the Mahalanobis turbulence of Kritzman and Li (2010)); Full, adding signal-level variables (the variance risk premium, the 21-day change in implied correlation, the RMT-cleaned signal). An unsupervised regime detector, a GMM and a Gaussian HMM restricted to filtered, forward-only probabilities (Rabiner, 1989), is fitted without the label under the same walk-forward and feeds its regime probability into the candidate pool. Validation is a purged walk-forward with expanding windows, a 63-day purge and a 21-day embargo, and the full protocol (features, label, hyperparameter grids, the net-Sharpe 0.42 bar to beat) was pre-registered before any result was computed.

4 Results

4.1 The correlation risk premium exists, and has compressed

Over the 7,221 trading days from 1996 to 2024, implied correlation exceeded the correlation that subsequently realised over the priced window by +0.079 on average (Newey–West $t = 7.4$ with 63 lags), and the premium was positive on 75% of days (Figure 1). The tradeable signal, built from trailing correlation, averages +0.078 ($t = 10.6$): correlation is persistent enough that the ex-ante proxy tracks the ex-post premium closely.

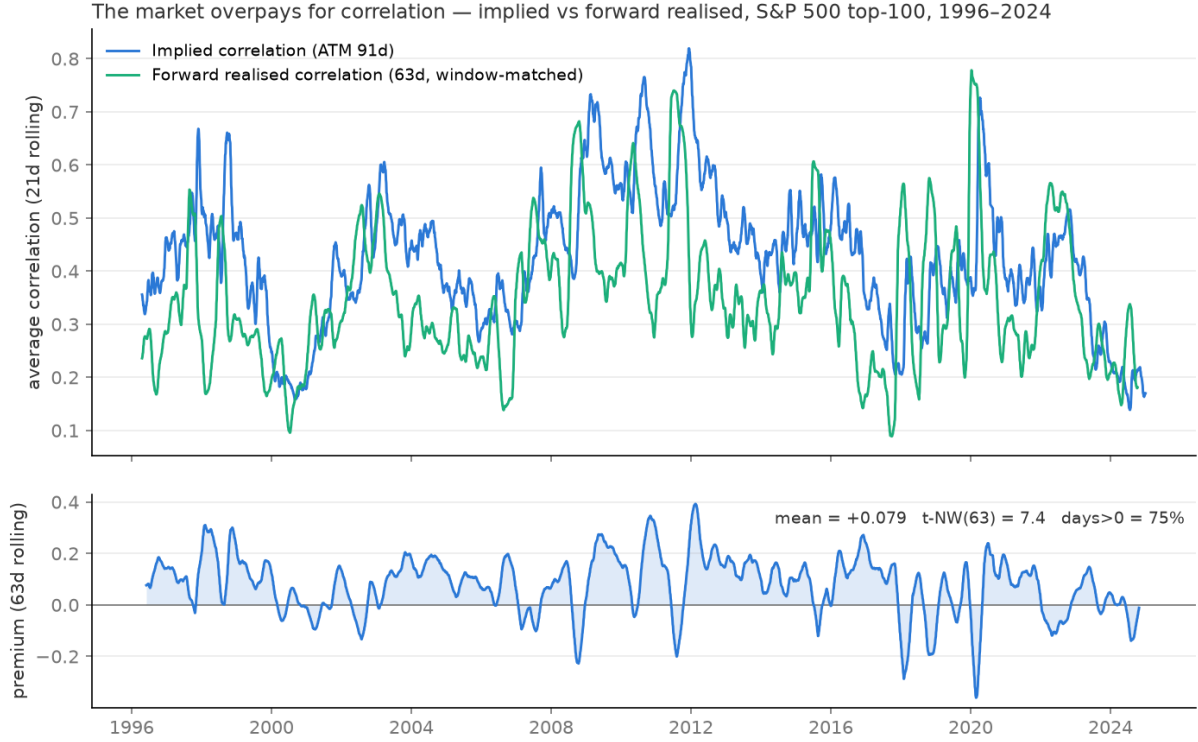


Figure 1: Implied versus forward realised correlation, S&P 500 top-100, 1996–2024, and the window-matched premium (mean +0.079, t -NW(63) = 7.4, positive on 75% of days).

The premium is not stable across time, though. Table 1 splits the sample: strong and significant through 2019, peaking at 0.135 in 2008–2012, then compressed to +0.028 ($t = 1.19$) in 2020–2024, indistinguishable from zero. We had registered this as a falsifiable limits-to-arbitrage prediction. DMV argue the premium persists because frictions stop outsiders from arbitraging it, so as spreads fell (our own cost anchors decline from 7.0% to 1.2% for large caps) the premium should shrink. It did.

4.2 The baseline trade: real, and nearly killed by frictions

The unconditional strategy v0 (trade every quarter, no signal) reproduces DMV’s Table II out of the box. The vega-neutral sizing puts the component leg at $\Sigma y = +99.5\%$ of wealth against their +101.12%; the gross Sharpe is 0.77 against their 0.73; the mean gross return is +7.3% per quarter ($t = 4.11$), positive in 70% of quarters, with skew -1.29 . Net of the

Table 1: Window-matched correlation premium by subperiod.

Period	Mean premium	t -NW(63)	% days > 0
1996–2003	0.085	4.87	73%
2004–2007	0.078	4.06	81%
2008–2012	0.135	4.25	83%
2013–2019	0.069	3.29	75%
2020–2024	0.028	1.19	63%

cost grid, their Table V replicates on our 29 years: the Sharpe falls to 0.42 (theirs 0.41), the mean return to +3.9% per quarter ($t = 2.24$), and the $\times 165$ gross cumulative becomes $\times 2.5$ (Figure 2). The premium survives costs, marginally.

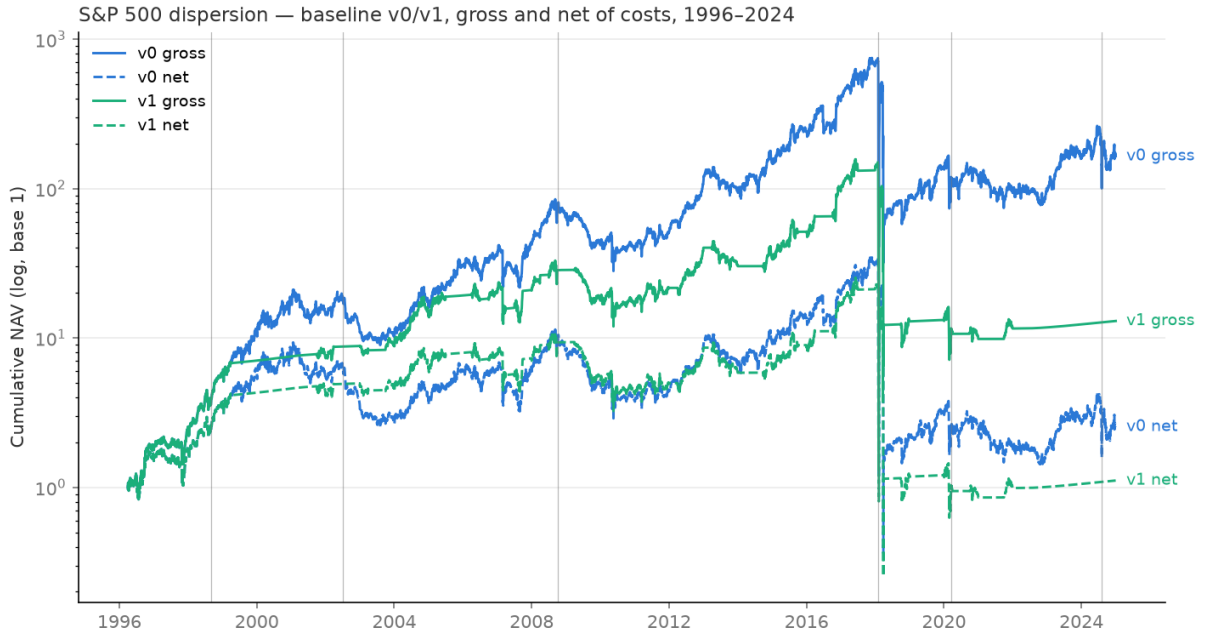


Figure 2: Cumulative NAV (log scale) of the unconditional v0 and signal-gated v1, gross and net of costs, 1996–2024.

The tail is the problem. Gross maximum drawdown is -95.8% , and one quarter, Q1-2018 (Volmageddon), loses 91% on its own. The daily ledger shows both option legs of that quarter ending positive; the loss came through the delta hedge, which bled -154% of wealth as realised correlation spiked. Attribution across the sample makes the same point: calm quarters earn on the short index leg ($+10.9\%$ on average), crisis quarters lose through the hedge (-23.0%), the realised-variance channel of eq. (3). A first conditioning attempt, v1, trades only when the signal is above its ex-ante median (58 of 115 quarters). It earns more per traded quarter ($+7.9\%$ against $+7.3\%$) and skips 2002, Lehman and 2024, but it trades Q1-2018, because the spread measures how rich the premium is, not how dangerous the quarter will be. Net, v1 barely survives ($\times 1.1$ cumulative): its traded quarters cluster in the wide-spread era.

4.3 RMT helps, but the value is the estimation window, not the clipping

Gating on the RMT-cleaned 252-day signal (`v1_rmt`) transforms the risk profile: net Sharpe 0.56, skew +0.80 (from -1.33), maximum drawdown -58.5% (from -99.0%), cumulative $\times 12.4$, and all three crashes avoided, including Volmageddon. Figure 3 shows the spectral geometry: with the Laloux-corrected edge, $K = 7$ eigenvalues escape the noise bulk (the market mode plus sector factors) where the naive edge finds 4.

252-day devolatilised spectrum vs Marchenko-Pastur — noise bulk and clipping edges

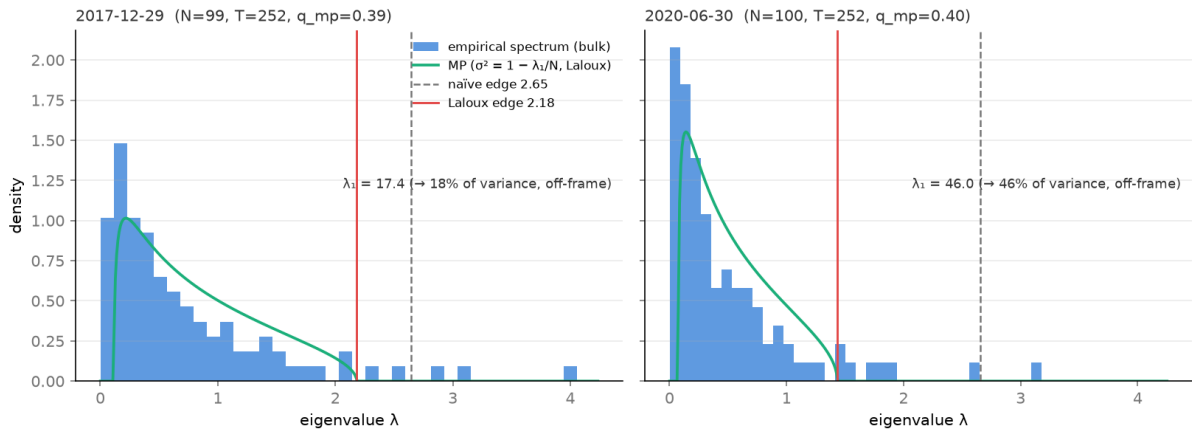


Figure 3: 252-day de-volatilised eigenvalue spectra against the Marchenko–Pastur density, with the naive and Laloux-corrected clipping edges, on a calm (2017) and a stressed (2020) date.

An ablation forces honesty here. Re-running the gate on the raw, un-clipped 252-day correlation gives virtually the same strategy: gross Sharpe 0.78, with the two gates disagreeing on 2 of 115 quarters; the clipping itself adds roughly +0.02 of Sharpe. The value comes from the slow 252-day window, which does not mistake a stressed quarter for an opportunity the way the reactive 63-day series does. We state this plainly because the fashionable claim would be the opposite.

The subperiod split (Figure 4) adds a second nuance: the gate’s advantage is regime-dependent, not uniform. It shines in 2004–2007 (net 1.01 against v_0 ’s 0.67), but v_0 wins in 2008–2012 (0.39 against 0.03) and in 2013–2019 (0.62 against 0.48): in the premium-rich crisis era, always trading did fine on Sharpe and gating merely cut trades. The gate’s genuine, sample-wide benefit is tail shape, earned by dodging specific crashes. The spectral structure also suggests an answer to frictions: trading only the top-20 names cuts the cost from 3.68% to 3.36% per traded quarter and beats the full basket net (Sharpe 0.595 against 0.571, skew +1.14) while losing slightly gross (0.75 against 0.78; `fig_parsimonious.png`). The effect is non-monotonic ($n = 30$ and $n = 50$ fall below the full basket), so the defensible claim is that 15–20 names do at least as well net at lower cost, not that 20 is optimal.

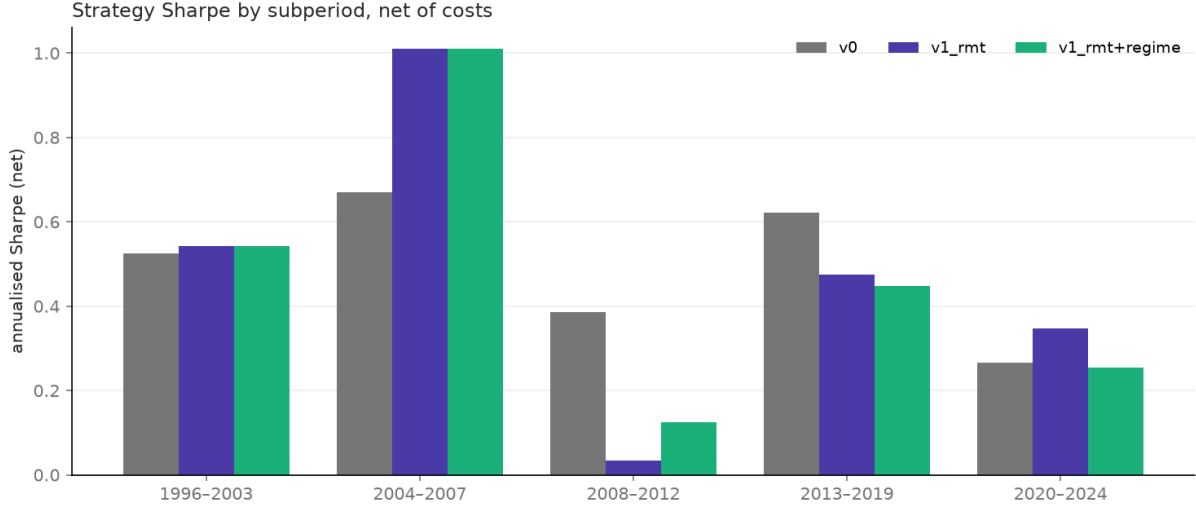


Figure 4: Net Sharpe by subperiod: v0, v1_rmt and v1_rmt+regime.

4.4 The ML timing layer is a null; a regime gate trims the tail

The supervised meta-model cannot predict the trade’s quarterly return. Out of sample, the correlation between prediction and outcome is +0.02 for the VIX-only feature set, −0.14 with spectral features added and −0.20 for the full set (`fig_pred_scatter.png`). The central test of the project is therefore answered: spectral features do not beat the VIX for timing this trade, because neither predicts it. Crisis recall is stark: the VIX model catches 3 of 8 crisis quarters, the spectral model none (Figure 5).

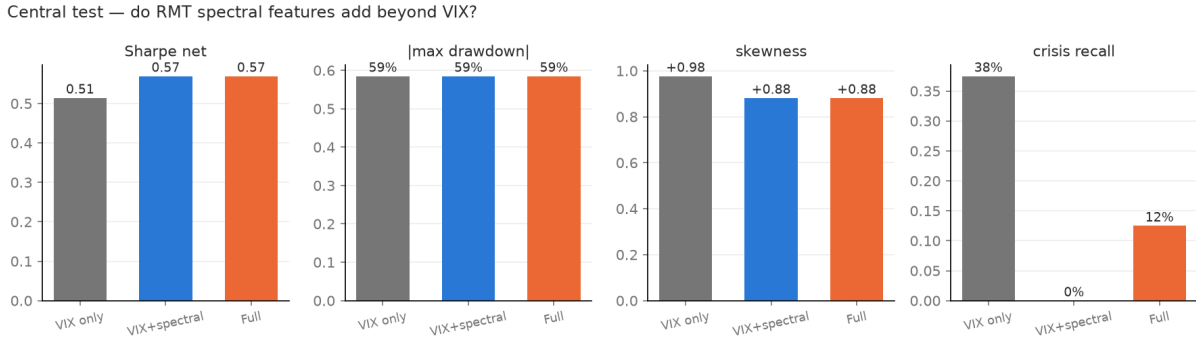


Figure 5: Central test: net Sharpe, drawdown, skew and crisis recall by feature set.

The veto built on these predictions changes nothing. It moves v1_rmt’s net Sharpe from 0.56 to 0.57, which is noise, and a VIX-only veto slightly hurts (0.51); Figure 6 shows the vetoed NAV overlapping v1_rmt almost everywhere.

The null is informative rather than embarrassing. The unsupervised regime detector fires in every crisis of the sample (Figure 7), so stress detection works. A follow-up experiment shows the label was the obstacle, not the features: switching the target to the daily forward correlation spike ($\bar{\rho}_{t+63} - \bar{\rho}_t$, roughly 7,000 observations instead of 90) lifts out-of-sample correlation to +0.40. Yet vetoing on that predictable spike still lowers the Sharpe (0.53 against 0.56). Detecting stress is not predicting the loss, because stress is

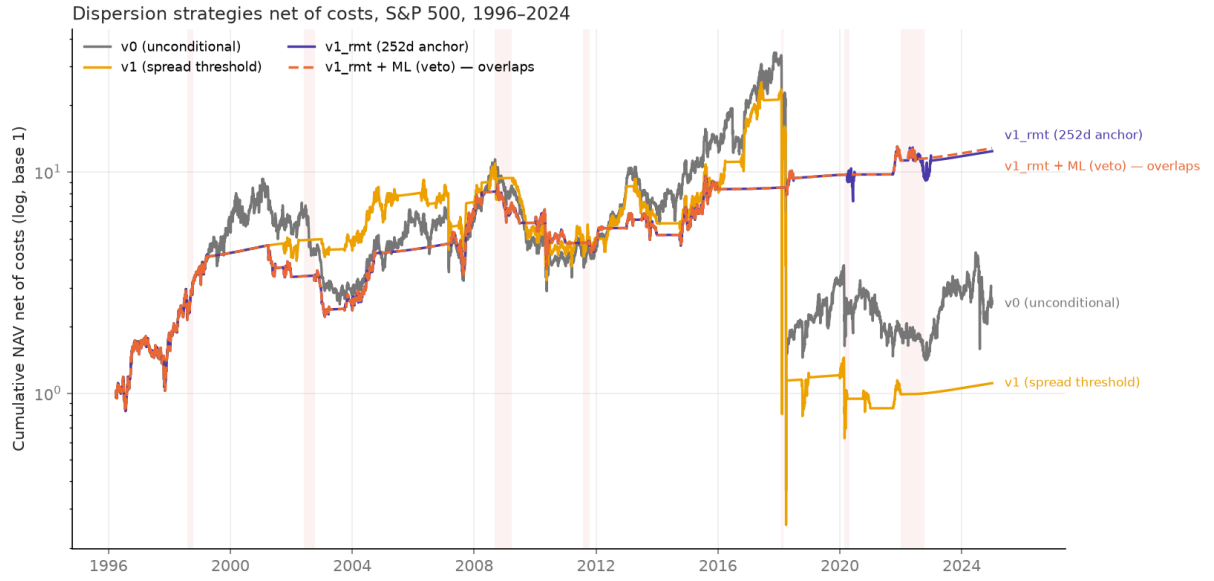


Figure 6: Net NAV of v0, v1, v1_rmt and v1_rmt+ML: the ML veto overlaps v1_rmt almost everywhere.

often exactly when the premium is richest; the danger appears to be priced.

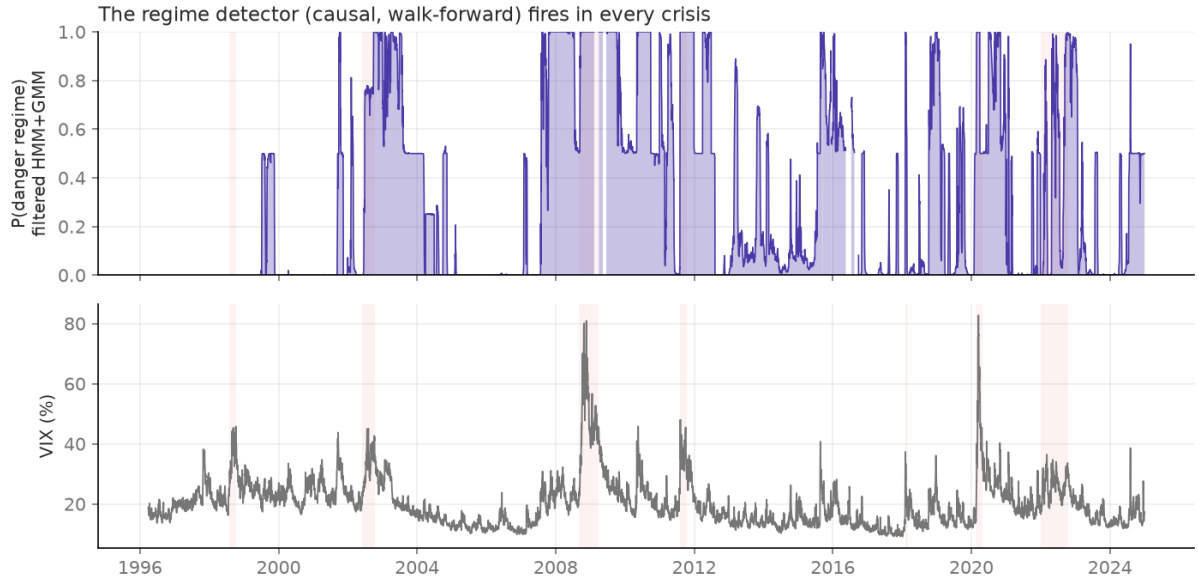


Figure 7: The causal (filtered, walk-forward) regime detector against the VIX: the danger probability rises in every crisis of the sample.

The one overlay that earns its place is the simplest: veto the trade when the causal regime probability sits in its high tail. It leaves the net Sharpe at 0.57 but lifts net skew to +1.00 and cuts the drawdown to -52.4% (Table 2). The improvement survives every threshold choice: net Sharpe stays within 0.46–0.67 across cost $\pm 50\%$ (above v0's 0.42 even at $\times 1.5$), within 0.57–0.58 across the regime-veto quantile and 0.54–0.56 across the gate quantile, with a tighter veto lifting skew to +1.31.

Last, the ATM proxy is quantifiably conservative: rebuilding the index leg's model-free

Table 2: Strategy summary, 1996–2024, 115 quarters.

Strategy	Net Sharpe	Net skew	Net maxDD	Gross Sharpe	Gross skew
v0 (unconditional)	0.42	−1.33	−99.0%	0.77	−1.29
v1_rmt	0.56	+0.80	−58.5%	0.80	+1.28
v1_rmt + regime	0.57	+1.00	−52.4%	0.78	+1.51

implied variance from the full smile raises the index vol by 1.2 points on average (19.7% against 18.5%), on all 116 rebalances, which lifts implied correlation by +0.066 (from 0.43 to 0.50; Figure 8). Measured with MFIV, the premium would be larger; our headline number is a lower bound.

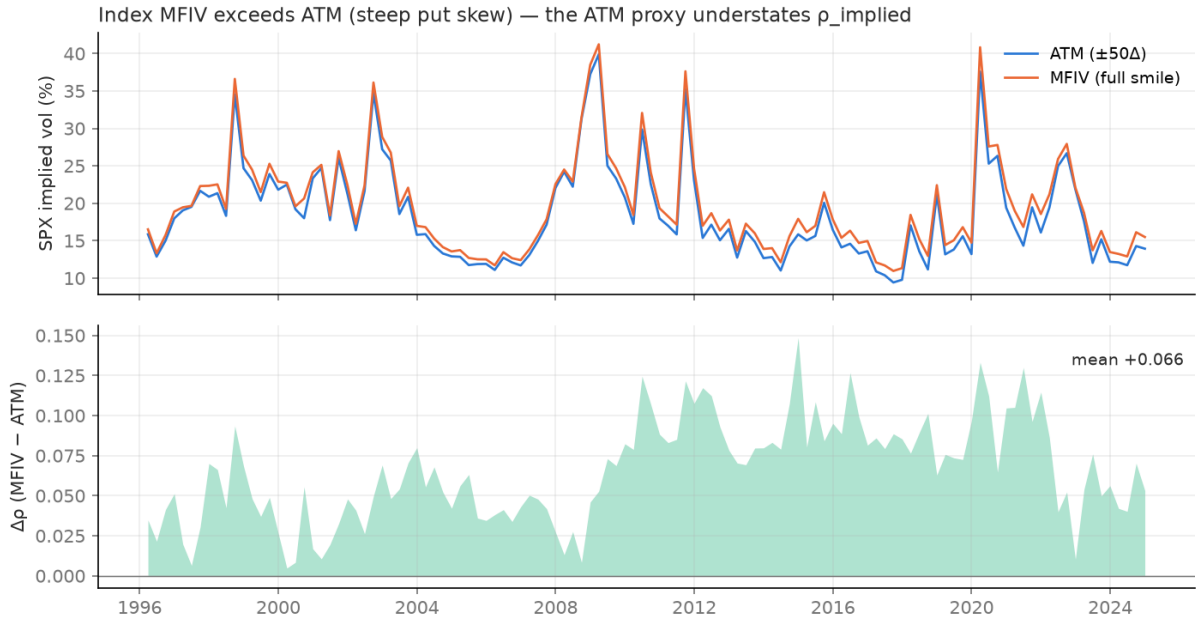


Figure 8: Index MFIV exceeds the ATM vol on every rebalance (mean gap 1.2 vol points); propagated through eq. (2), the ATM proxy understates implied correlation by 0.066 on average.

5 Limitations

Several approximations deserve plain statement. Weights use full market capitalisation rather than free float (the float-weight file sits outside our subscription) and are renormalised over the top-100 basket, so the implied-correlation measure embeds an index-versus-basket basis; it is, at least, the same basis the book itself trades. The ATM proxy for implied volatility understates the premium, and Section 4.4 quantifies the bias at +0.066 of implied correlation and signs it in our favour, so the headline number is a lower bound. Costs are a calibrated parametric grid fitted to quoted spreads, not per-trade quotes; quoted spreads overstate effective ones, which is again conservative, but the grid remains a model. The engine holds every position to maturity, with no intra-quarter unwind or stop-loss; an early-exit variant could shape the drawdowns differently, and straddles are path-dependent where variance swaps are not. Both are design choices we accept and document rather than test. Vega neutrality is exact only at inception: as spot and volatility drift, Volga and Vanna grow and the neutrality erodes (the v0 book's total vega drifted by roughly 3,600 over Q1-2018; `fig_vanna_volga.png`), part of why crisis quarters bleed. The ML null is honest but low-powered: about 90 quarterly predictions containing 12 crises cannot support strong claims, and the daily-label experiment suggests the target, not the features, was the binding constraint. The RMT scalar effect is small by construction, since clipping preserves the market mode that dominates $\bar{\rho}$. Finally, a 17-day OptionMetrics surface outage (August 2020) is accepted as missing data rather than repaired.

6 Conclusion

The correlation risk premium is real. Over 29 years, S&P 500 option prices charged +0.079 more correlation than subsequently realised, and a DMV-style vega-neutral dispersion book converted it into a gross Sharpe of 0.77. Almost everything else in this project qualifies that sentence. Frictions take the net Sharpe to 0.42; the premium compressed to statistical insignificance after 2020, exactly as a limits-to-arbitrage account predicts; and the fashionable layers helped less, and differently, than advertised. RMT improved the strategy through the humble 252-day estimation window, not the eigenvalue clipping. Supervised ML could not predict the trade's return at all, and even a predictable correlation spike did not help, because the danger is priced. What survives is deliberately simple: gate on the slow cleaned signal, veto in detected stress regimes, trade fewer names. Net Sharpe 0.57, positive skew, and drawdowns cut roughly in half: modest, honest and defensible.

References

- Anthropic. (2026) *Claude* [Large language model]. Available from: <https://claude.ai/> [Accessed on various dates between 5 May and 25 July 2026].
- Bakshi, G. & Kapadia, N. (2003) Delta-hedged gains and the negative market volatility risk premium. *The Review of Financial Studies*. 16 (2), 527–566.
- Bakshi, G., Kapadia, N. & Madan, D. (2003) Stock return characteristics, skew laws, and the differential pricing of individual equity options. *The Review of Financial Studies*. 16 (1), 101–143.
- Bun, J., Bouchaud, J.-P. & Potters, M. (2017) Cleaning large correlation matrices: tools from random matrix theory. *Physics Reports*. 666, 1–109.
- Carr, P. & Madan, D. (1998) Towards a theory of volatility trading. In: Jarrow, R. (ed.) *Volatility: new estimation techniques for pricing derivatives*. London, Risk Books. pp. 417–427.
- Center for Research in Security Prices (CRSP). (2025) *CRSP US stock and index databases*. Accessed via Wharton Research Data Services, <https://wrds-www.wharton.upenn.edu/> [Accessed: 20 June 2026].
- Driessen, J., Maenhout, P. J. & Vilkov, G. (2009) The price of correlation risk: evidence from equity options. *The Journal of Finance*. 64 (3), 1377–1406.
- Google. (2026) *Gemini* [Large language model]. Available from: <https://gemini.google.com/> [Accessed on various dates between 12 May and 3 July 2026].
- Kritzman, M. & Li, Y. (2010) Skulls, financial turbulence, and risk management. *Financial Analysts Journal*. 66 (5), 30–41.
- Kritzman, M., Li, Y., Page, S. & Rigobon, R. (2011) Principal components as a measure of systemic risk. *The Journal of Portfolio Management*. 37 (4), 112–126.
- Laloux, L., Cizeau, P., Bouchaud, J.-P. & Potters, M. (1999) Noise dressing of financial correlation matrices. *Physical Review Letters*. 83 (7), 1467–1470.
- López de Prado, M. (2018) *Advances in financial machine learning*. Hoboken, NJ, John Wiley & Sons.
- Marchenko, V. A. & Pastur, L. A. (1967) Distribution of eigenvalues for some sets of random matrices. *Mathematics of the USSR-Sbornik*. 1 (4), 457–483.
- OpenAI. (2026) *ChatGPT* [Large language model]. Available from: <https://chatgpt.com/> [Accessed on various dates between 8 May and 21 July 2026].
- OptionMetrics. (2025) *IvyDB US options database*. Accessed via Wharton Research Data Services, <https://wrds-www.wharton.upenn.edu/> [Accessed: 20 June 2026].
- Perplexity AI. (2026) *Perplexity* [AI search engine]. Available from: <https://www.perplexity.ai/> [Accessed on various dates between 19 May and 14 July 2026].

- Potters, M. & Bouchaud, J.-P. (2020) *A first course in random matrix theory: for physicists, engineers and data scientists*. Cambridge, Cambridge University Press.
- Rabiner, L. R. (1989) A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*. 77 (2), 257–286.
- Sebbag, N. (2026) *Dispersion-Trading-Project* [source code repository]. GitHub. Available from: <https://github.com/NSB22/Dispersion-Trading-Project> [Accessed: 25 July 2026].

A Repository guide

The full implementation (code, notebooks, figures and tables) lives in a single repository, <https://github.com/NSB22/Dispersion-Trading-Project> (Sebbag, 2026). This appendix explains how it is organised and how to reproduce every number in the paper; Appendix B then lists the source code in full.

A.1 Structure

The layout separates the reproducible pipeline (`main.py`, `src/`, `tests/`) from the exploratory work (`exploration_notebooks/`) and the generated outputs (`data/`, `results/`):

<code>main.py</code>	end-to-end pipeline: WRDS -> parquets -> backtests
<code>pyproject.toml</code> , <code>uv.lock</code>	locked dependency stack (uv, Python 3.12)
<code>plan.md</code>	pre-registered ML protocol (written before results)
<code>README.md</code>	full project documentation
<code>src/dispersion/</code>	
<code>data/</code>	WRDS access, universe, IV, returns, spots, assembly
<code>wrds_client.py</code> <code>universe.py</code> <code>iv.py</code> <code>returns.py</code> <code>spots.py</code> <code>assemble.py</code>	
<code>signal/</code>	
<code>implied_corr.py</code>	implied correlation, premium and signal
<code>rmt/</code>	
<code>cleaning.py</code> <code>daily.py</code>	Marchenko-Pastur + Laloux cleaning; daily spectra
<code>backtest/</code>	
<code>marking.py</code> <code>engine.py</code>	daily mark-to-surface; the dispersion engine
<code>ml/</code>	
<code>features.py</code> <code>regime.py</code> <code>metamodel.py</code> <code>experiments.py</code>	
<code>utils/</code>	
<code>greeks.py</code>	BS/Black-76 pricing, greeks, relative conversions
<code>tests/</code>	pytest suite (37 tests)
<code>exploration_notebooks/</code>	01-10: exploration later promoted into <code>src/</code>
<code>data/{raw,processed}</code>	parquet deliverables (git-ignored, WRDS licensing)
<code>results/{figures,tables}</code>	every figure and table in this paper

The working rule throughout the project was to explore in notebooks (surface conventions in 01, CRSP membership in 02, the signal in 03-04, costs in 05, backtests in 06, the spectrum in 07, ML in 08-09, robustness in 10), then promote validated logic into `src/`. The final backtest runs from the `.py` modules only, so no result depends on notebook state.

A.2 Setup

The project uses `uv` as package manager, with Python 3.12 and every dependency pinned in `pyproject.toml/uv.lock`. Raw data is never committed (WRDS licensing), so reproduction requires a WRDS account with CRSP and OptionMetrics access. Setup is three steps:

```
uv sync                                # installs Python 3.12 + the locked stack
cp .env.example .env                  # then set WRDS_USERNAME and PYTHONPATH=src
uv run --env-file .env python -m dispersion.data.wrds_client # test access
```

PYTHONPATH must point to `src/` so the `dispersion` package is importable; the WRDS password is prompted once and can be cached in a `.pgpass` outside the repository.

A.3 Running the pipeline

A single entry point reproduces the paper end to end:

```
uv run --env-file .env python main.py          # full run (WRDS pull included)
uv run --env-file .env python main.py --no-data # reuse the base parquets
uv run pytest                                  # test suite, 37 tests
```

The full run pulls WRDS, writes the base parquet deliverables to `data/processed/` (implied volatilities, weights, realised series, marking surface, spots, rates, returns), builds the signal and the ML features, then backtests `v0`, `v1`, `v1_rmt` and `v1_rmt+regime`, gross and net of costs. Figures and tables land in `results/`, under the exact filenames referenced in this paper, so each exhibit can be traced back to the code that produced it.

B Source code

This appendix lists the full implementation. The project is organised as a Python package (`src/dispersion`) with a top-level pipeline script (`main.py`) and a pytest suite (`tests/`). Code is grouped below by pipeline stage: data acquisition, signal construction, RMT cleaning, the backtest engine, the ML layer, shared utilities, then tests.

B.1 Pipeline entry point

Listing 1: `main.py` – end-to-end pipeline

```
1  """
2  Run the whole project end to end: pull WRDS, build the base parquets, fit the
3  signals and ML features, then backtest v0 / v1 / v1_rmt / v1_rmt+regime. This is
4  the reproducible path from raw WRDS to the strategy P&L; notebooks only read the
5  parquets it writes.
6
7      uv run --env-file .env python main.py          # full run
8      uv run --env-file .env python main.py --no-data # skip the WRDS rebuild, reuse
9      base parquets
10 """
11 import argparse
12 import time
13
14 import numpy as np
15 import pandas as pd
16
17 from dispersion.backtest.engine import exante_quantile_threshold, run_backtest
18 from dispersion.data.assemble import build_dataset
19 from dispersion.data.wrds_client import get_connection
20 from dispersion.ml.experiments import _high_veto, _stats
21 from dispersion.ml.features import build_ml_dataset
22 from dispersion.ml.regime import build_regime_feature
23 from dispersion.rmt.daily import build_rmt_daily, build_signal_rmt
24 from dispersion.signal.implied_corr import build_signal
25
26 PROC = "data/processed"
27
28 def stage_data():
29     db = get_connection()
30     out = build_dataset(db)
31     db.close()
32     return {k: v.shape for k, v in out.items()}
33
34
35 def stage_models():
36     build_signal()          # signal.parquet
37     build_rmt_daily()       # rmt_daily.parquet: 252d cleaned rho-bar + spectral
38                             # features
```

```

38 build_signal_rmt()      # signal_rmt.parquet: the RMT-cleaned signal
39 build_ml_dataset()     # ml_dataset.parquet: features + spike label
40 build_regime_feature() # adds f_reg_gmm / f_reg_hmm / f_regime to ml_dataset
41
42
43 def stage_backtests():
44     signal = pd.read_parquet(f"{PROC}/signal.parquet")
45     signal_rmt = pd.read_parquet(f"{PROC}/signal_rmt.parquet")
46     ml = pd.read_parquet(f"{PROC}/ml_dataset.parquet")
47     weights = pd.read_parquet(f"{PROC}/weights.parquet")
48     for d in (signal, signal_rmt, ml):
49         d["date"] = pd.to_datetime(d["date"])
50     weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
51     rebals = sorted(weights["rebalance_date"].unique())
52     base = {k: pd.read_parquet(f"{PROC}/{k}.parquet") for k in ("surface", "spots", "
rates", "weights")}
53
54     def bt(sig, thr, net, tag):
55         r = run_backtest(data={**{k: v.copy() for k, v in base.items()}}, "signal": sig.
copy()),
56                             threshold=thr, costs=({} if net else None))
57         suffix = "net" if net else "gross"
58         r["quarterly"].to_parquet(f"{PROC}/mainrun_{tag}_{suffix}_quarterly.parquet",
index=False)
59         return _stats(r["quarterly"], r["daily"])
60
61     # gates: v1 on the base signal, v1_rmt on the RMT-cleaned signal, both ex-ante
median
62     thr_v1 = exante_quantile_threshold(signal, rebals, q=0.5, warmup=12)
63     thr_v1rmt = exante_quantile_threshold(signal_rmt, rebals, q=0.5, warmup=12)
64     reg_at = {pd.Timestamp(r): float(ml.loc[ml["date"] == pd.Timestamp(r), "f_regime"].
iloc[0])
65               for r in rebals if (ml["date"] == pd.Timestamp(r)).any()
66               and np.isfinite(ml.loc[ml["date"] == pd.Timestamp(r), "f_regime"].iloc
[0]))}
67     thr_reg = thr_v1rmt.copy()
68     for d in _high_veto(reg_at, q=0.67):
69         thr_reg[pd.Timestamp(d)] = 1e9
70
71     res = {}
72     res["v0_gross"] = bt(signal, None, False, "v0")
73     res["v0_net"] = bt(signal, None, True, "v0")
74     res["v1_net"] = bt(signal, thr_v1, True, "v1")
75     res["v1rmt_gross"] = bt(signal_rmt, thr_v1rmt, False, "v1rmt")
76     res["v1rmt_net"] = bt(signal_rmt, thr_v1rmt, True, "v1rmt")
77     res["v1rmt_regime_gross"] = bt(signal_rmt, thr_reg, False, "v1rmt_regime")
78     res["v1rmt_regime_net"] = bt(signal_rmt, thr_reg, True, "v1rmt_regime")
79     return res
80
81
82 def main():

```

```

83 ap = argparse.ArgumentParser()
84 ap.add_argument("--no-data", action="store_true", help="skip the WRDS rebuild")
85 args = ap.parse_args()
86
87 t0 = time.time()
88 if not args.no_data:
89     print("[1/3] data - building base parquets from WRDS ...", flush=True)
90     shapes = stage_data()
91     print("      " + " | ".join(f"{k} {v}" for k, v in shapes.items()), flush=True
92 )
93 else:
94     print("[1/3] data - skipped (reusing base parquets)", flush=True)
95
96 print("[2/3] models - signal, RMT, ML features, regime ...", flush=True)
97 stage_models()
98
99 print("[3/3] backtests - v0 / v1 / v1_rmt / v1_rmt+regime ...", flush=True)
100 res = stage_backtests()
101 print(f"\n{'strategy':22s} {'Sharpe':>7s} {'skew':>6s} {'maxDD':>7s} {'cumul':>7s}
102 ", flush=True)
103 for k, s in res.items():
104     print(f"{k:22s} {s['sharpe']:+7.2f} {s['skew']:+6.2f} {s['maxDD']:>7.1%} {s['
105 cumul']:>6.1f}x", flush=True)
106 print(f"\ndone in {time.time() - t0:.0f}s", flush=True)
107
108 if __name__ == "__main__":
109     main()

```

B.2 Data acquisition (src/dispersion/data/)

Listing 2: src/dispersion/data/wrds_client.py

```

1  """
2  WRDS connection plumbing.
3
4  Usage:
5      from dispersion.data.wrds_client import get_connection
6      db = get_connection()
7  """
8  import os
9  from dotenv import load_dotenv
10 import wrds
11
12 load_dotenv()
13
14
15 def get_connection() -> wrds.Connection:
16     """Open a WRDS connection (username read from .env)."""
17     username = os.getenv("WRDS_USERNAME")
18     if not username:

```

```

19         raise RuntimeError("WRDS_USERNAME missing. Add it to the .env file.")
20     return wrds.Connection(wrds_username=username)
21
22
23 def test_connection() -> None:
24     """Smoke test: check access to crsp, optionm, comp libraries."""
25     db = get_connection()
26     print("WRDS connection established.")
27     libs = db.list_libraries()
28     for lib in ["crsp", "optionm", "comp"]:
29         print(f" - {lib}: {'OK' if lib in libs else 'ABSENT'}")
30     db.close()
31
32
33 if __name__ == "__main__":
34     test_connection()

```

Listing 3: src/dispersion/data/universe.py

```

1  """
2  Point-in-time S&P 500 universe construction.
3
4  Usage:
5      from dispersion.data.universe import get_universe
6      df = get_universe(db, "2020-03-31")
7
8  """
9  import pandas as pd
10 import wrds
11
12 def get_universe(db: wrds.Connection, date: str, n: int = 100) -> pd.DataFrame:
13     """Top-N S&P 500 constituents by market cap on 'date', point-in-time.
14
15     Only names with a score-1 OptionMetrics link are kept, so each permno maps
16     to exactly one secid.
17
18     Columns: permno, secid, market_cap (USD on 'date'), weight (cap weight
19     renormalised within the top-N), rnk (rank by cap, 1 = largest).
20     """
21     query = f"""
22     WITH members AS (
23         -- point-in-time S&P 500 membership
24         SELECT permno
25         FROM crsp.dsp500list
26         WHERE start <= '{date}' AND ending >= '{date}'
27     ),
28     capi AS (
29         -- market cap = abs(prc) * shroul * 1000
30         -- CRSP prc can be negative (it's a bid/ask midpoint)
31         SELECT d.permno,
32                ABS(d.prc) * d.shroul * 1000 AS market_cap

```

```

33         FROM crsp.dsf d
34         JOIN members m ON d.permno = m.permno
35         WHERE d.date = '{date}'
36             AND d.prc IS NOT NULL
37             AND d.shrout IS NOT NULL
38     ),
39     ranked AS (
40         -- ROW_NUMBER not RANK: with RANK a cap tie at rank N would let in >N rows.
41         -- permno breaks ties deterministically.
42         SELECT permno, market_cap,
43             ROW_NUMBER() OVER (ORDER BY market_cap DESC, permno) AS rnk
44         FROM capi
45     ),
46     top_n AS (
47         SELECT * FROM ranked WHERE rnk <= {n}
48     ),
49     link AS (
50         -- score=1 only. A permno can have several overlapping score-1 links;
51         -- keep the latest edate so secid is unique per permno.
52         SELECT DISTINCT ON (permno) permno, secid
53         FROM wrdsapps_link_crsp_optionm.opcrsphist
54         WHERE score = 1 AND sdate <= '{date}' AND edate >= '{date}'
55         ORDER BY permno, edate DESC
56     )
57     SELECT t.permno,
58         l.secid,
59         t.market_cap,
60         t.market_cap / SUM(t.market_cap) OVER () AS weight,
61         t.rnk
62     FROM top_n t
63     LEFT JOIN link l ON t.permno = l.permno
64     ORDER BY t.rnk
65     """
66     return db.raw_sql(query)

```

Listing 4: src/dispersion/data/iv.py

```

1     """
2     Implied-volatility extraction from OptionMetrics 'vsurfd' (volatility surface).
3
4     Usage:
5         from dispersion.data.iv import get_iv, get_surface
6         df = get_iv(db, secids=[108105, 101594], date_start="2020-01-01", date_end="
7             2020-12-31")
8     """
9
10    import numpy as np
11    import pandas as pd
12    import wrds
13
14    def get_iv(

```



```

14 db: wrds.Connection,
15 secids: list[int],
16 date_start: str,
17 date_end: str,
18 days: int = 91,
19 ) -> pd.DataFrame:
20     """ATM (delta  $\pm 50$ ) implied vol at a fixed maturity for the given securities.
21
22     Pure extraction -- no rebalancing or membership logic. Works the same for the
23     index (secid 108105) and its constituents.
24
25     One row per (secid, date): secid, date, iv_call_50 (delta +50), iv_put_50
26     (delta -50), iv_atm (mean of the two legs, NaN if either is missing).
27     """
28     secid_list = ",".join(str(int(s)) for s in secids)
29     y0, y1 = int(date_start[:4]), int(date_end[:4])
30
31     frames = []
32     for year in range(y0, y1 + 1):
33         # clip the window to this year's slice of vsurfd
34         lo = max(date_start, f"{year}-01-01")
35         hi = min(date_end, f"{year}-12-31")
36         q = f"""
37         SELECT secid, date, delta, impl_volatility
38         FROM optionm.vsurfd{year}
39         WHERE secid IN ({secid_list})
40             AND days = {days}
41             AND delta IN (50, -50)
42             AND date BETWEEN '{lo}' AND '{hi}'
43             AND impl_volatility IS NOT NULL
44             AND impl_volatility > 0
45         """
46         frames.append(db.raw_sql(q))
47
48     long = pd.concat(frames, ignore_index=True)
49     if long.empty:
50         return pd.DataFrame(columns=["secid", "date", "iv_call_50", "iv_put_50", "
51 iv_atm"])
52
53     # pivot the two legs (delta +50 -> call, -50 -> put) into columns
54     wide = (
55         long.pivot_table(index=["secid", "date"], columns="delta", values="
56 impl_volatility")
57         .rename(columns={50.0: "iv_call_50", -50.0: "iv_put_50"})
58         .reset_index()
59     )
60     wide.columns.name = None
61     # make sure both leg columns exist even if a delta never showed up.
62     # use np.nan, not pd.NA -- an object column makes mean(skipna=False) raise
63     for col in ("iv_call_50", "iv_put_50"):
64         if col not in wide.columns:

```

```

63         wide[col] = np.nan
64
65     # need both legs or it's NaN
66     wide["iv_atm"] = wide[["iv_call_50", "iv_put_50"]].mean(axis=1, skipna=False)
67
68     return wide[["secid", "date", "iv_call_50", "iv_put_50", "iv_atm"]].sort_values(
69         ["secid", "date"]
70     ).reset_index(drop=True)
71
72
73 def get_surface(
74     db: wrds.Connection,
75     secids: list[int],
76     date_start: str,
77     date_end: str,
78     days: tuple[int, ...] = (10, 30, 60, 91),
79 ) -> pd.DataFrame:
80     """Standardised surface (both  $\pm 50\Delta$  legs) at several maturity pillars.
81
82     Marking-grade data: a 91d position ages into shorter residual maturities, so
83     we pull the sub-91 pillars too. Raw vendor values, no cleaning here.
84
85     One row per (secid, date, days, cp_flag): secid, date, days, cp_flag
86     ('C'/'P'), iv, premium, strike.
87     """
88     secid_list = ",".join(str(int(s)) for s in secids)
89     days_list = ",".join(str(int(d)) for d in days)
90     y0, y1 = int(date_start[:4]), int(date_end[:4])
91
92     frames = []
93     for year in range(y0, y1 + 1):
94         lo = max(date_start, f"{year}-01-01")
95         hi = min(date_end, f"{year}-12-31")
96         q = f"""
97         SELECT secid, date, days, cp_flag,
98                impl_volatility AS iv, impl_premium AS premium, impl_strike AS strike
99         FROM optionm.vsurfd{year}
100        WHERE secid IN ({secid_list})
101              AND days IN ({days_list})
102              AND delta IN (50, -50)
103              AND date BETWEEN '{lo}' AND '{hi}'
104              AND impl_volatility IS NOT NULL
105              AND impl_volatility > 0
106        """
107         frames.append(db.raw_sql(q))
108
109     surf = pd.concat(frames, ignore_index=True)
110     if surf.empty:
111         return pd.DataFrame(columns=["secid", "date", "days", "cp_flag", "iv", "
112         premium", "strike"])

```

```

113 surf["cp_flag"] = surf["cp_flag"].str.strip()
114 surf["days"] = surf["days"].astype(int)
115 return surf.sort_values(["secid", "date", "days", "cp_flag"]).reset_index(drop=
True)

```

Listing 5: src/dispersion/data/returns.py

```

1 """
2 Returns extraction from CRSP 'dsf' and realized-volatility computation.
3
4 Usage:
5     from dispersion.data.returns import get_returns, realized_vol
6     px = get_returns(db, permnos=[14593, 10107], date_start="2019-01-01", date_end="
2020-12-31")
7     vol = realized_vol(px)          # {21: df, 63: df}, annualised
8 """
9 import numpy as np
10 import pandas as pd
11 import wrds
12
13 TRADING_DAYS = 252
14
15
16 def get_returns(
17     db: wrds.Connection,
18     permnos: list[int],
19     date_start: str,
20     date_end: str,
21 ) -> pd.DataFrame:
22     """Daily total returns (dividend- and split-adjusted) for the given permnos.
23
24     CRSP 'ret' is a holding-period return, so dividends and splits are already
25     baked in -- no manual price adjustment.
26
27     Wide DataFrame: index = date, columns = permno, values = daily return; NaN
28     where a security did not trade.
29     """
30     permno_list = ",".join(str(int(p)) for p in permnos)
31     q = f"""
32     SELECT permno, date, ret
33     FROM crsp.dsf
34     WHERE permno IN ({permno_list})
35         AND date BETWEEN '{date_start}' AND '{date_end}'
36         AND ret IS NOT NULL
37     """
38     long = db.raw_sql(q)
39     if long.empty:
40         return pd.DataFrame()
41
42     wide = long.pivot(index="date", columns="permno", values="ret").sort_index()
43     wide.columns.name = None

```

```

44     return wide
45
46
47 def realized_vol(
48     returns: pd.DataFrame,
49     windows: tuple[tuple[int, int], ...] = ((21, 17), (63, 50)),
50 ) -> dict[int, pd.DataFrame]:
51     """Rolling annualised realized vol per security, one DataFrame per window.
52
53     Uses log-returns; a window is NaN until it has at least 'min_obs'
54     observations. 'windows' is a tuple of (length, min_obs) pairs in trading days.
55     Returns dict {window_length: wide DataFrame (date x permno)}.
56     """
57     log_ret = np.log1p(returns)
58     out = {}
59     for window, min_obs in windows:
60         vol = log_ret.rolling(window=window, min_periods=min_obs).std(ddof=1)
61         out[window] = vol * np.sqrt(TRADING_DAYS)
62     return out
63
64
65 def realized_corr_matrix(
66     returns: pd.DataFrame,
67     end_date: str,
68     window: int = 63,
69     min_obs: int = 50,
70     method: str = "listwise",
71 ) -> pd.DataFrame | None:
72     """Realized correlation matrix over the 'window' days ending at 'end_date'.
73
74     'method' is 'listwise' (complete-case rows -> PSD) or 'pairwise'. Returns an
75     N x N DataFrame (index/columns = permno), or None if there isn't enough data.
76     """
77     log_ret = np.log1p(returns)
78     win = log_ret.loc[:end_date].tail(window)
79     # drop names with too few obs in the window
80     win = win.loc[:, win.notna().sum() >= min_obs]
81     if win.shape[1] < 2:
82         return None
83
84     if method == "listwise":
85         win = win.dropna(axis=0, how="any") # keep only complete rows so it stays PSD
86         if len(win) < min_obs:
87             return None
88         corr = win.corr()
89     elif method == "pairwise":
90         corr = win.corr(min_periods=min_obs)
91     else:
92         raise ValueError(f"method must be 'listwise' or 'pairwise', got {method!r}")
93     return corr
94

```

```

95
96 def average_correlation(
97     corr: pd.DataFrame,
98     weights: pd.Series,
99     vols: pd.Series,
100     method: str = "weighted",
101 ) -> float:
102     """Collapse a correlation matrix into a single average correlation rho-bar.
103
104     'weighted' (default) is the formula-consistent average from the index
105     variance decomposition (matches the implied-correlation definition):
106         rho_bar = (v' R v - sum v_i^2) / ((sum v_i)^2 - sum v_i^2), v_i = w_i *
107     sigma_i
108     'equal' is the plain mean of off-diagonal pairwise correlations.
109
110     'weights' are the frozen rebalance cap weights; 'vols' are realized sigma_i on
111     the window's end date. Returns NaN if not computable.
112     """
113     permnos = corr.index
114     w = weights.reindex(permnos)
115     s = vols.reindex(permnos)
116     valid = w.notna() & s.notna()
117     permnos = permnos[valid]
118     if len(permnos) < 2:
119         return float("nan")
120
121     R = corr.loc[permnos, permnos].to_numpy()
122
123     if method == "weighted":
124         v = (w[permnos] * s[permnos]).to_numpy()
125     elif method == "equal":
126         v = np.ones(len(permnos))
127     else:
128         raise ValueError(f"method must be 'weighted' or 'equal', got {method!r}")
129
130     sum_sq = np.sum(v**2)
131     num = v @ R @ v - sum_sq # off-diagonal weighted sum of rho_ij
132     den = v.sum() ** 2 - sum_sq # off-diagonal weighted sum (rho_ij = 1)
133     return float(num / den) if den != 0 else float("nan")

```

Listing 6: src/dispersion/data/spots.py

```

1 """Daily spot prices from OptionMetrics 'secprd'.
2
3 'secprd' is the price series OptionMetrics builds the surface from, so its closes
4 share the split convention of 'impl_strike'. Marking and intrinsic settlement
5 need S and K on the same convention (|S - K|); mid-quarter splits are handled
6 with 'cfadj'.
7
8 Usage:
9     from dispersion.data.spots import get_spots

```

```

10     df = get_spots(db, [108105], "2020-01-01", "2020-12-31")
11 """
12 import pandas as pd
13 import wrds
14
15
16 def get_spots(
17     db: wrds.Connection,
18     secids: list[int],
19     date_start: str,
20     date_end: str,
21 ) -> pd.DataFrame:
22     """Close price + cumulative split-adjustment factor per (secid, date)."""
23     secid_list = ",".join(str(int(s)) for s in secids)
24     y0, y1 = int(date_start[:4]), int(date_end[:4])
25
26     frames = []
27     for year in range(y0, y1 + 1):
28         lo = max(date_start, f"{year}-01-01")
29         hi = min(date_end, f"{year}-12-31")
30         q = f"""
31         SELECT secid, date, close, cfadj
32         FROM optionm.secprd{year}
33         WHERE secid IN ({secid_list})
34             AND date BETWEEN '{lo}' AND '{hi}'
35             AND close IS NOT NULL
36         """
37         frames.append(db.raw_sql(q))
38
39     spots = pd.concat(frames, ignore_index=True)
40     if spots.empty:
41         return pd.DataFrame(columns=["secid", "date", "close", "cfadj"])
42
43     spots["close"] = spots["close"].abs() # close can be signed (midpoint convention);
44     take abs
45     return spots.sort_values(["secid", "date"]).reset_index(drop=True)

```

Listing 7: src/dispersion/data/assemble.py

```

1  """Build the clean backtest dataset: pull each piece, align to one calendar,
2  apply the cleaning rules, and write the parquets to data/processed/.
3
4  Usage:
5      from dispersion.data.assemble import build_dataset
6      out = build_dataset(db, "1996-01-01", "2024-12-31")
7  """
8  import os
9  import numpy as np
10 import pandas as pd
11 import wrds
12

```

```

13 from .universe import get_universe
14 from .iv import get_iv, get_surface
15 from .spots import get_spots
16 from .returns import (
17     get_returns,
18     realized_vol,
19     realized_corr_matrix,
20     average_correlation,
21 )
22
23 SPX_SECID = 108105
24
25
26 # ----- #
27 # Calendar helpers
28 # ----- #
29 def _master_calendar(db, iv_index: pd.DataFrame, date_start: str, date_end: str) -> pd.
    DatetimeIndex:
30     """Strict intersection of OptionMetrics (SPX IV) and CRSP (dsf) trading days."""
31     iv_dates = pd.to_datetime(iv_index["date"]).unique()
32     dsf = db.raw_sql(
33         f"SELECT DISTINCT date FROM crsp.dsf WHERE date BETWEEN '{date_start}' AND '{
            date_end}'"
34     )
35     dsf_dates = pd.to_datetime(dsf["date"]).unique()
36     common = pd.DatetimeIndex(sorted(set(iv_dates) & set(dsf_dates)))
37     return common
38
39
40 def _rebalance_dates(calendar: pd.DatetimeIndex) -> list[pd.Timestamp]:
41     """Last trading day of each quarter present in the master calendar."""
42     s = pd.Series(calendar, index=calendar)
43     grp = s.groupby([calendar.year, calendar.quarter]).max()
44     return [pd.Timestamp(x) for x in grp.values]
45
46
47 # ----- #
48 # Main
49 # ----- #
50 def build_dataset(
51     db: wrds.Connection,
52     date_start: str = "1996-01-01",
53     date_end: str = "2024-12-31",
54     n: int = 100,
55     iv_days: int = 91,
56     corr_window: int = 63,
57     corr_min_obs: int = 50,
58     corr_method: str = "listwise",
59     rho_method: str = "weighted",
60     ffill_limit: int = 3,
61     iv_pct: tuple[float, float] = (0.001, 0.999),

```

```

62     surface_days: tuple[int, ...] = (10, 30, 60, 91),
63     returns_start: str = "1995-01-01",
64     out_dir: str | None = "data/processed",
65 ) -> dict[str, pd.DataFrame]:
66     """Build and optionally save the clean backtest dataset; returns a dict of
67     DataFrames.
68
69     Writes iv_index, iv_components, weights, realized_vol, realized_corr,
70     corr_matrices (cleaned, signal-grade) plus surface, rates, returns (marking/
71     RMT grade, stored raw). 'returns_start' predates 'date_start' so the 252-day
72     RMT windows have warm history from the first rebalance.
73     """
74
75     # --- 1. SPX IV (the index leg) + master calendar + rebalance schedule -----
76     iv_index = get_iv(db, [SPX_SECID], date_start, date_end, days=iv_days)
77     iv_index["date"] = pd.to_datetime(iv_index["date"])
78     calendar = _master_calendar(db, iv_index, date_start, date_end)
79     rebals = _rebalance_dates(calendar)
80
81     iv_index = iv_index[iv_index["date"].isin(calendar)].drop(columns="secid")
82     # bounded ffill on the index series; no outlier clipping on the index
83     iv_index = (
84         iv_index.set_index("date").reindex(calendar).ffill(limit=ffill_limit).
85         reset_index(names="date")
86     )
87
88     # --- 2. Per-quarter loop ----- #
89     weights_rows, iv_comp_parts, vol_parts, rho_rows, corr_mat_parts = [], [], [], [],
90     []
91     surface_parts, spot_parts = [], []
92
93     # active window per rebalance; the cleaning step reuses this too
94     active_by_reb: dict[pd.Timestamp, pd.DatetimeIndex] = {}
95     for i, reb in enumerate(rebals):
96         nxt = rebals[i + 1] if i + 1 < len(rebals) else calendar[-1] + pd.Timedelta(
97             days=1)
98         active_by_reb[reb] = calendar[(calendar >= reb) & (calendar < nxt)]
99
100     for reb in rebals:
101         active = active_by_reb[reb]
102         if len(active) == 0:
103             continue
104
105         univ = get_universe(db, str(reb.date()), n=n)
106         univ["permno"] = univ["permno"].astype(int)
107         univ["secid"] = univ["secid"].astype("Int64")
108         w = univ.set_index("permno")["weight"]
109         permnos = univ["permno"].tolist()
110         secids = univ["secid"].dropna().astype(int).tolist()
111         sec2permno = dict(zip(univ["secid"].astype("Int64"), univ["permno"]))

```



```

109     # weights (frozen) at this rebalance
110     wr = univ[["permno", "secid", "weight", "market_cap", "rnk"]].copy()
111     wr.insert(0, "rebalance_date", reb)
112     weights_rows.append(wr)
113
114     # component IV over the active window (point-in-time secids)
115     ivc = get_iv(db, secids, str(active[0].date()), str(active[-1].date()), days=
iv_days)
116     if not ivc.empty:
117         ivc["date"] = pd.to_datetime(ivc["date"])
118         ivc["permno"] = ivc["secid"].astype("Int64").map(sec2permno).astype("Int64
")
119         ivc["rebalance_date"] = reb
120         iv_comp_parts.append(ivc[["rebalance_date", "date", "permno", "secid", "
iv_atm"]])
121
122     # multi-pillar surface (marking-grade, stored raw) for this quarter's names
123     surf = get_surface(db, secids, str(active[0].date()), str(active[-1].date()),
days=surface_days)
124     if not surf.empty:
125         surf["date"] = pd.to_datetime(surf["date"])
126         surf = surf[surf["date"].isin(active)]
127         surf["permno"] = surf["secid"].astype("Int64").map(sec2permno).astype("
Int64")
128         surf.insert(0, "rebalance_date", reb)
129         surface_parts.append(surf)
130
131     # daily spots (same split convention as the strikes)
132     sp = get_spots(db, secids, str(active[0].date()), str(active[-1].date()))
133     if not sp.empty:
134         sp["date"] = pd.to_datetime(sp["date"])
135         sp = sp[sp["date"].isin(active)]
136         sp["permno"] = sp["secid"].astype("Int64").map(sec2permno).astype("Int64")
137         sp.insert(0, "rebalance_date", reb)
138         spot_parts.append(sp)
139
140     # returns with a trailing buffer for the rolling windows
141     buf_start = (active[0] - pd.Timedelta(days=corr_window * 2 + 20)).date()
142     ret = get_returns(db, permnos, str(buf_start), str(active[-1].date()))
143     if ret.empty:
144         continue
145     ret.index = pd.to_datetime(ret.index)
146     vols = realized_vol(ret)
147
148     # realized vol (long) on active dates
149     v21 = vols[21].reindex(active)
150     v63 = vols[63].reindex(active)
151     vlong = (
152         v21.reset_index(names="date").melt("date", var_name="permno", value_name="
vol_21")
153         .merge(

```

```

154         v63.reset_index(names="date").melt("date", var_name="permno",
value_name="vol_63"),
155         on=["date", "permno"],
156     )
157 )
158 vol_parts.append(vlong)
159
160 # daily rho-bar + full matrix at the rebalance date
161 for d in active:
162     C = realized_corr_matrix(ret, str(d.date()), corr_window, corr_min_obs,
corr_method)
163     if C is None:
164         rho_rows.append((d, np.nan, np.nan))
165         continue
166     if d not in vols[63].index: # skip rather than fall back to a stale vol
row
167         rho_rows.append((d, np.nan, np.nan))
168         continue
169     vols_d = vols[63].loc[d]
170     rho_rows.append(
171         (
172             d,
173             average_correlation(C, w, vols_d, rho_method),
174             average_correlation(C, w, vols_d, "equal"),
175         )
176     )
177     if d == reb: # store the full matrix only at rebalances
178         m = C.stack().rename("corr").reset_index()
179         m.columns = ["permno_i", "permno_j", "corr"]
180         m.insert(0, "rebalance_date", reb)
181         corr_mat_parts.append(m)
182
183 # --- 2bis. Marking/RMT-grade extras: SPX surface, zero curve, returns panel #
184 surf_spx = get_surface(db, [SPX_SECID], date_start, date_end, days=surface_days)
185 surf_spx["date"] = pd.to_datetime(surf_spx["date"])
186 surf_spx = surf_spx[surf_spx["date"].isin(calendar) & (surf_spx["date"] >= rebals
[0])].copy()
187 reb_idx = pd.DatetimeIndex(rebals)
188 surf_spx.insert(
189     0, "rebalance_date",
190     reb_idx[reb_idx.searchsorted(surf_spx["date"].values, side="right") - 1],
191 )
192 surf_spx["permno"] = pd.Series(pd.NA, index=surf_spx.index, dtype="Int64")
193
194 surface = pd.concat(surface_parts + [surf_spx], ignore_index=True)
195 surface["secid"] = surface["secid"].astype("Int64")
196 surface = surface[
197     ["rebalance_date", "date", "permno", "secid", "days", "cp_flag", "iv", "
premium", "strike"]
198 ].sort_values(["rebalance_date", "date", "secid", "days", "cp_flag"]).reset_index(
drop=True)

```

```

199
200 spx_spots = get_spots(db, [SPX_SECID], date_start, date_end)
201 spx_spots["date"] = pd.to_datetime(spx_spots["date"])
202 spx_spots = spx_spots[spx_spots["date"].isin(calendar) & (spx_spots["date"] >=
203 rebals[0])].copy()
204 spx_spots.insert(
205     0, "rebalance_date",
206     reb_idx[reb_idx.searchsorted(spx_spots["date"].values, side="right") - 1],
207 )
208 spx_spots["permno"] = pd.Series(pd.NA, index=spx_spots.index, dtype="Int64")
209
210 spots = pd.concat(spot_parts + [spx_spots], ignore_index=True)
211 spots["secid"] = spots["secid"].astype("Int64")
212 spots = spots[
213     ["rebalance_date", "date", "permno", "secid", "close", "cfadj"]
214 ].sort_values(["rebalance_date", "date", "secid"]).reset_index(drop=True)
215
216 rates = db.raw_sql(
217     f"SELECT date, days, rate FROM optionm.zerocd WHERE date BETWEEN '{date_start}'
218     AND '{date_end}'"
219 )
220 rates["date"] = pd.to_datetime(rates["date"])
221 rates = rates[rates["date"].isin(calendar)].sort_values(["date", "days"]).
222 reset_index(drop=True)
223
224 # union of all constituents ever held; history from returns_start for 252d RMT
225 windows
226 all_permnos = sorted({int(p) for wr in weights_rows for p in wr["permno"]})
227 ret_all = get_returns(db, all_permnos, returns_start, date_end)
228 ret_all.index = pd.to_datetime(ret_all.index)
229 returns_long = (
230     ret_all.reset_index(names="date")
231     .melt("date", var_name="permno", value_name="ret")
232     .dropna(subset=["ret"])
233     .sort_values(["date", "permno"])
234     .reset_index(drop=True)
235 )
236 returns_long["permno"] = returns_long["permno"].astype(int)
237
238 # --- 3. Cleaning of component IV (global percentile outliers, then ffill) -- #
239 iv_components = pd.concat(iv_comp_parts, ignore_index=True)
240 lo, hi = iv_components["iv_atm"].quantile(list(iv_pct))
241 iv_components.loc[(iv_components["iv_atm"] < lo) | (iv_components["iv_atm"] > hi),
242     "iv_atm"] = np.nan
243 # ffill per (quarter, permno) over the full active window, not just up to the
244 # name's last observation -- so trailing gaps stay NaN, extended at most
245 ffill_limit days
246 cleaned = []
247 for (reb, _), g in iv_components.groupby(["rebalance_date", "permno"]):
248     active = active_by_reb[reb]
249     s = g.set_index("date")["iv_atm"].reindex(active).ffill(limit=ffill_limit)

```

```

244         cleaned.append(
245             pd.DataFrame(
246                 {"date": s.index, "permno": g["permno"].iloc[0], "secid": g["secid"].
                iloc[0],
247                 "iv_atm": s.values, "rebalance_date": reb}
248             )
249         )
250     iv_components = pd.concat(cleaned, ignore_index=True)[
251         ["rebalance_date", "date", "permno", "secid", "iv_atm"]
252     ]
253
254     # --- 4. Collect ----- #
255     out = {
256         "iv_index": iv_index,
257         "iv_components": iv_components,
258         "weights": pd.concat(weights_rows, ignore_index=True),
259         "realized_vol": pd.concat(vol_parts, ignore_index=True),
260         "realized_corr": pd.DataFrame(rho_rows, columns=["date", "rho_bar", "
rho_bar_equal"]),
261         "corr_matrices": pd.concat(corr_mat_parts, ignore_index=True),
262         "surface": surface,
263         "spots": spots,
264         "rates": rates,
265         "returns": returns_long,
266     }
267
268     if out_dir:
269         os.makedirs(out_dir, exist_ok=True)
270         for name, df in out.items():
271             df.to_parquet(os.path.join(out_dir, f"{name}.parquet"), index=False)
272
273     return out

```

B.3 Signal construction (src/dispersion/signal/)

Listing 8: src/dispersion/signal/implied_corr.py

```

1  """
2  Implied correlation, the correlation-risk premium, and the dispersion signal.
3
4  Inverts the one-factor index-variance identity on ATM 91-day IVs to get an
5  implied rho, then builds two series against the realised rho-bar:
6
7      premium(t) = rho_implied(t) - rho_trailing(t + horizon)    # ex-post check
8      signal(t)  = rho_implied(t) - rho_trailing(t)              # ex-ante, tradeable
9
10 Usage:
11     from dispersion.signal.implied_corr import build_signal
12     sig = build_signal()    # reads data/processed/, writes signal.parquet
13 """
14 import os

```

```

15
16 import numpy as np
17 import pandas as pd
18
19
20 def implied_correlation(
21     iv_components: pd.DataFrame,
22     weights: pd.DataFrame,
23     iv_index: pd.DataFrame,
24     n_min: int = 90,
25 ) -> pd.DataFrame:
26     """
27     Daily implied correlation from the one-factor inversion.
28
29      $\rho_{\text{implied}} = (\sigma_I^2 - S2) / (S1^2 - S2)$ , with  $S1 = \sum(w_{\text{hat}} * \sigma)$ 
30     and  $S2 = \sum(w_{\text{hat}}^2 * \sigma^2)$  over the names with a valid IV that day;
31      $w_{\text{hat}}$  renormalises the frozen rebalance weights over that set.
32
33      $n_{\text{min}}$  sets a coverage floor: days with fewer valid names are dropped.
34     Returns a frame indexed by date with  $[\rho_{\text{implied}}, n_{\text{names}}]$ .
35     """
36     m = iv_components.merge(
37         weights[["rebalance_date", "permno", "weight"]],
38         on=["rebalance_date", "permno"],
39         how="left",
40         validate="m:1",
41     )
42     if m["weight"].isna().any():
43         raise ValueError("component IV rows without a matching universe weight")
44
45     valid = m.dropna(subset=["iv_atm"]).copy()
46     valid["w_sig"] = valid["weight"] * valid["iv_atm"]
47     valid["w2_sig2"] = valid["weight"] ** 2 * valid["iv_atm"] ** 2
48
49     g = valid.groupby("date").agg(
50         w_sum=("weight", "sum"),
51         s1_raw=("w_sig", "sum"),
52         s2_raw=("w2_sig2", "sum"),
53         n_names=("weight", "size"),
54     )
55     s1 = g["s1_raw"] / g["w_sum"] # renormalised weighted-average component IV
56     s2 = g["s2_raw"] / g["w_sum"] ** 2 # renormalised diagonal term
57
58     sig_i = iv_index.set_index("date")["iv_atm"].reindex(g.index)
59     n_missing = int(sig_i.isna().sum())
60     if n_missing: # a NaN here would look like a coverage-floor day, not an input
61         mismatch
62         raise ValueError(f"{n_missing} component-IV dates have no index IV --
63         inconsistent inputs")
64     rho = (sig_i ** 2 - s2) / (s1 ** 2 - s2)
65     rho[g["n_names"] < n_min] = np.nan # coverage floor

```

```

64
65     return pd.DataFrame(
66         {"rho_implied": rho.astype("float64"), "n_names": g["n_names"].astype("Int64")}
67     )
68
69
70 def build_signal(
71     processed_dir: str = "data/processed",
72     horizon: int = 63,
73     n_min: int = 90,
74     out_file: str | None = "signal.parquet",
75 ) -> pd.DataFrame:
76     """
77     Build the full signal table and optionally write it to parquet.
78
79     rho_forward(t) is rho_trailing shifted back 'horizon' rows so it lines up
80     with the implied window. Columns: date, rho_implied, rho_trailing,
81     rho_forward, premium, signal, n_names.
82     """
83     iv_index = pd.read_parquet(os.path.join(processed_dir, "iv_index.parquet"))
84     iv_comp = pd.read_parquet(os.path.join(processed_dir, "iv_components.parquet"))
85     weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
86     rcorr = pd.read_parquet(os.path.join(processed_dir, "realized_corr.parquet"))
87
88     iv_comp["secid"] = iv_comp["secid"].astype("Int64")
89     iv_comp["permno"] = iv_comp["permno"].astype("Int64")
90     weights["permno"] = weights["permno"].astype("Int64")
91     for df in (iv_index, iv_comp, weights, rcorr):
92         for c in ("date", "rebalance_date"):
93             if c in df.columns:
94                 df[c] = pd.to_datetime(df[c])
95
96     rho = implied_correlation(iv_comp, weights, iv_index, n_min=n_min)
97
98     spine = rcorr.set_index("date").sort_index() # master calendar
99     lost = rho.index.difference(spine.index)
100     extra = spine.index.difference(rho.index)
101     if len(lost) or len(extra): # the right-join must be lossless
102         raise ValueError(
103             f"calendar mismatch: {len(lost)} rho dates off-spine, {len(extra)} spine
104             dates without rho"
105         )
106     sig = rho.join(spine[["rho_bar"]], how="right").sort_index()
107     sig = sig.rename(columns={"rho_bar": "rho_trailing"})
108     sig["rho_forward"] = sig["rho_trailing"].shift(-horizon)
109     sig["premium"] = sig["rho_implied"] - sig["rho_forward"]
110     sig["signal"] = sig["rho_implied"] - sig["rho_trailing"]
111
112     cols = ["rho_implied", "rho_trailing", "rho_forward", "premium", "signal"]
113     sig[cols] = sig[cols].astype("float64")

```

```

113 sig["n_names"] = sig["n_names"].astype("Int64")
114 sig = sig.reset_index(names="date")[["date"] + cols + ["n_names"]]
115
116 if out_file:
117     sig.to_parquet(os.path.join(processed_dir, out_file), index=False)
118 return sig

```

B.4 Random matrix theory cleaning (src/dispersion/rmt/)

Listing 9: src/dispersion/rmt/cleaning.py

```

1 """
2 RMT cleaning of realised correlation matrices: EWMA de-vol, a 252-day
3 correlation of the standardised residuals, then Marchenko-Pastur clipping of
4 the noise bulk (Laloux edge).
5
6 Here 'q_mp' = N/T is the MP aspect ratio, not the dividend yield 'q' used in
7 the pricing modules.
8
9 Usage:
10     from dispersion.rmt.cleaning import devolatilise, corr_window, laloux_clip
11     z = devolatilise(returns_wide)
12     C = corr_window(z, date, permnos)
13     C_clean, diag = laloux_clip(C)
14 """
15 import numpy as np
16 import pandas as pd
17
18 T_WIN = 252
19 MIN_OBS = 200
20 EWMA_LAMBDA = 0.94
21
22
23 def devolatilise(returns_wide: pd.DataFrame, lam: float = EWMA_LAMBDA,
24                 min_periods: int = 20) -> pd.DataFrame:
25     """
26     Standardised residuals z_t = log-return_t / sigma_{t-1}, sigma from an EWMA.
27     The one-day shift keeps sigma predictable so correlation windows don't peek
28     ahead.
29     """
30     logret = np.log1p(returns_wide.astype("float64"))
31     sigma = logret.ewm(alpha=1.0 - lam, min_periods=min_periods).std().shift(1)
32     return logret / sigma
33
34
35 def corr_window(z: pd.DataFrame, date, permnos, t_win: int = T_WIN,
36               min_obs: int = MIN_OBS, min_names: int = 60):
37     """
38     Complete-case correlation of the last 't_win' rows of z up to 'date' for the
39     given universe. Names with fewer than 'min_obs' observations are dropped
40     first. Returns None if the panel is too thin, else (C, t_eff).

```

```

41
42     t_eff is the number of complete-case rows actually used; the MP edge needs
43     q_mp = N/t_eff, not N/t_win, or iid noise slips through as signal.
44     """
45     cols = [p for p in permnos if p in z.columns]
46     win = z.loc[:pd.Timestamp(date), cols].tail(t_win)
47     win = win.dropna(axis=1, thresh=min_obs).dropna(axis=0)
48     if win.shape[1] < min_names or win.shape[0] < min_obs:
49         return None
50     return win.corr(), int(win.shape[0])
51
52
53 def laloux_clip(C: pd.DataFrame, t_win: int = T_WIN):
54     """
55     Clip eigenvalues below the Laloux edge to the bulk mean, then renormalise.
56     Returns (C_clean, diagnostics); diagnostics holds n, t, q_mp, lam1_share,
57     edge_naive, edge_laloux, k_signal and trace_prenorm.
58     """
59     is_df = isinstance(C, pd.DataFrame)
60     A = C.to_numpy(dtype="float64") if is_df else np.asarray(C, dtype="float64")
61     n = A.shape[0]
62     ev, V = np.linalg.eigh(A)                                # ascending
63     q_mp = n / t_win
64     lam1_share = ev[-1] / n
65     edge_naive = (1.0 + np.sqrt(q_mp)) ** 2
66     edge = (1.0 - lam1_share) * edge_naive                    # Laloux correction
67     bulk = ev < edge
68     ev2 = ev.copy()
69     if bulk.any():
70         ev2[bulk] = ev[bulk].mean()                          # trace-preserving on the bulk
71     A2 = (V * ev2) @ V.T
72     d = np.sqrt(np.diag(A2))
73     A2 = A2 / np.outer(d, d)
74     np.fill_diagonal(A2, 1.0)
75
76     out = pd.DataFrame(A2, index=C.index, columns=C.columns) if is_df else A2
77     diagnostics = {
78         "n": n, "t": t_win, "q_mp": q_mp, "lam1_share": float(lam1_share),
79         "edge_naive": float(edge_naive), "edge_laloux": float(edge),
80         "k_signal": int((~bulk).sum()), "trace_prenorm": float(ev2.sum()),
81     }
82     return out, diagnostics
83
84
85 def spectral_features(C: pd.DataFrame, t_win: int = T_WIN, top: int = 5):
86     """
87     Regime features of a correlation matrix. Returns (features dict, dominant
88     eigenvector); the caller uses v1 for the day-to-day rotation.
89
90     Features: lam1_share (market mode), k_signal (factors above the Laloux
91     edge), absorption_top (top-'top' absorption ratio), lam2_share, spec_entropy

```



```

92     (spectral entropy, risk concentration), pr_v1 (how spread the market mode is).
93     """
94     A = C.to_numpy(dtype="float64")
95     n = A.shape[0]
96     ev, V = np.linalg.eigh(A)
97     q_mp = n / t_win
98     edge = (1.0 - ev[-1] / n) * (1.0 + np.sqrt(q_mp)) ** 2
99     p = np.clip(ev, 1e-12, None) / n
100    v1 = pd.Series(V[:, -1], index=C.index)
101    if v1.sum() < 0:                                     # sign convention: market mode
102        v1 = -v1                                         positive
103    feats = {
104        "lam1_share": float(ev[-1] / n),
105        "k_signal": int((ev >= edge).sum()),
106        "absorption_top": float(ev[-top:].sum() / n),
107        "lam2_share": float(ev[-2] / n) if n >= 2 else np.nan,
108        "spec_entropy": float(-(p * np.log(p)).sum()),
109        "pr_v1": float(1.0 / (v1.to_numpy() ** 4).sum() / n),
110    }
111    return feats, v1

```

Listing 10: src/dispersion/rmt/daily.py

```

1  """
2  Daily RMT series: a cleaned rho-bar variant of the signal plus spectral regime
3  features for the ML layer. Builds rmt_daily.parquet and signal_rmt.parquet.
4
5  signal_rmt keeps a 'premium' column only to match signal.parquet's schema --
6  don't use it. A 252d trailing leg shifted by 63 rows overlaps itself ~75%, so
7  that premium is mechanically contaminated; only 'signal' is safe to trade on.
8
9  Usage:
10     from dispersion.rmt.daily import build_rmt_daily, build_signal_rmt
11     build_rmt_daily()      # ~7,200 window eigendecompositions, a few minutes
12     build_signal_rmt()
13 """
14 import os
15
16 import numpy as np
17 import pandas as pd
18
19 from ..data.returns import average_correlation
20 from .cleaning import (EWMA_LAMBDA, MIN_OBS, T_WIN, corr_window, devolatilise,
21                        laloux_clip, spectral_features)
22
23
24 def _rotation(v1: pd.Series, v1_prev: pd.Series | None) -> float:
25     """
26     Day-to-day rotation of the dominant eigenvector:  $1 - |\langle v1_t, v1_{t-1} \rangle|$  over
27     the common names (each subvector renormalised). NaN with no previous vector

```

```

28     or no overlap.
29     """
30     if v1_prev is None:
31         return np.nan
32     common = v1.index.intersection(v1_prev.index)
33     if len(common) < 10:
34         return np.nan
35     a = v1.loc[common].to_numpy(dtype="float64")
36     b = v1_prev.loc[common].to_numpy(dtype="float64")
37     na, nb = np.linalg.norm(a), np.linalg.norm(b)
38     if na == 0 or nb == 0:
39         return np.nan
40     return float(1.0 - abs(a @ b) / (na * nb))
41
42
43 def build_rmt_daily(
44     processed_dir: str = "data/processed",
45     t_win: int = T_WIN,
46     min_obs: int = MIN_OBS,
47     lam: float = EWMA_LAMBDA,
48     out_file: str | None = "rmt_daily.parquet",
49 ) -> pd.DataFrame:
50     """Daily cleaned rho-bar + spectral features, frozen universe per quarter."""
51     returns = pd.read_parquet(os.path.join(processed_dir, "returns.parquet"))
52     weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
53     signal = pd.read_parquet(os.path.join(processed_dir, "signal.parquet"))
54
55     returns["date"] = pd.to_datetime(returns["date"])
56     weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
57     spine = pd.DatetimeIndex(pd.to_datetime(signal["date"])).sort_values()
58
59     ret = returns.pivot(index="date", columns="permno", values="ret").sort_index().
60     astype("float64")
61     z = devolatilise(ret, lam=lam)
62     sig252 = (np.log1p(ret)).rolling(t_win, min_periods=min_obs).std() * np.sqrt
63     (252.0)
64
65     rebals = sorted(weights["rebalance_date"].unique())
66     rows = []
67     for k, reb in enumerate(rebals):
68         reb = pd.Timestamp(reb)
69         nxt = pd.Timestamp(rebals[k + 1]) if k + 1 < len(rebals) else spine[-1] + pd.
70         Timedelta(days=1)
71         active = spine[(spine >= reb) & (spine < nxt)]
72         wq = weights[weights["rebalance_date"] == reb]
73         wmap = wq.set_index("permno")["weight"].astype("float64")
74         v1_prev, lam1_prev = None, np.nan
75
76         for d in active:
77             res = corr_window(z, d, wmap.index.tolist(), t_win=t_win, min_obs=min_obs)
78             if res is None:

```

```

76         rows.append((d,) + (np.nan,) * 12 + (0,))
77         v1_prev, lam1_prev = None, np.nan
78         continue
79     C, t_eff = res # effective T for the MP edge
80     C_clean, diag = laloux_clip(C, t_win=t_eff)
81     feats, v1 = spectral_features(C, t_win=t_eff)
82
83     kept = C.columns
84     w_kept = wmap.reindex(kept)
85     s_kept = sig252.loc[d, kept] # KeyError if d is missing,
rather than a stale row
86     rho_raw = average_correlation(C, w_kept, s_kept, "weighted")
87     rho_clean = average_correlation(C_clean, w_kept, s_kept, "weighted")
88
89     # cross-name dispersion of average correlation
90     A = C.to_numpy(dtype="float64")
91     corr_xdisp = float(np.std(A.mean(axis=1)))
92     # Mahalanobis turbulence on the cleaned inverse -- inverting a noisy C
93     # is where cleaning actually matters
94     zrow = z.loc[d, kept].to_numpy(dtype="float64")
95     mask = np.isfinite(zrow)
96     if mask.sum() >= 30:
97         Ainv = np.linalg.pinv(C_clean.to_numpy(dtype="float64")[np.ix_(mask,
mask)])
98         zc = zrow[mask]
99         turb = float(zc @ Ainv @ zc / mask.sum())
100     else:
101         turb = np.nan
102
103     rows.append((d, rho_raw, rho_clean, feats["lam1_share"], feats["k_signal"],
104
105         feats["absorption_top"],
106         feats["lam1_share"] - lam1_prev if np.isfinite(lam1_prev)
107     else np.nan,
108         _rotation(v1, v1_prev), feats["lam2_share"], feats["
109 spec_entropy"],
110         feats["pr_v1"], corr_xdisp, turb, len(kept)))
111     v1_prev, lam1_prev = v1, feats["lam1_share"]
112
113 out = pd.DataFrame(rows, columns=["date", "rho_rmt_raw", "rho_rmt_clean", "
114 lam1_share",
115
116         "k_signal", "absorption_top", "d_lam1", "
117 rotation",
118         "lam2_share", "spec_entropy", "pr_v1", "
119 corr_xdisp",
120         "turb", "n_names_rmt"])
121
122 if out_file:
123     out.to_parquet(os.path.join(processed_dir, out_file), index=False)
124 return out
125
126
127
128

```

```

119 def build_signal_rmt(
120     processed_dir: str = "data/processed",
121     horizon: int = 63,
122     out_file: str | None = "signal_rmt.parquet",
123 ) -> pd.DataFrame:
124     """
125     Signal variant whose trailing realised leg is the RMT-cleaned 252d rho-bar.
126     Same columns as signal.parquet so the engine gate works unchanged.
127     """
128     sig = pd.read_parquet(os.path.join(processed_dir, "signal.parquet"))
129     rmt = pd.read_parquet(os.path.join(processed_dir, "rmt_daily.parquet"))
130     sig["date"] = pd.to_datetime(sig["date"])
131     rmt["date"] = pd.to_datetime(rmt["date"])
132
133     m = sig[["date", "rho_implied", "n_names"]].merge(
134         rmt[["date", "rho_rmt_clean", "n_names_rmt"]], on="date", how="left", validate
135         ="1:1"
136     ).sort_values("date").reset_index(drop=True)
137     m["rho_trailing"] = m["rho_rmt_clean"].astype("float64")
138     m["rho_forward"] = m["rho_trailing"].shift(-horizon)
139     m["premium"] = m["rho_implied"] - m["rho_forward"]
140     m["signal"] = m["rho_implied"] - m["rho_trailing"]
141     m = m[["date", "rho_implied", "rho_trailing", "rho_forward", "premium", "signal",
142           "n_names", "n_names_rmt"]]
143     if out_file:
144         m.to_parquet(os.path.join(processed_dir, out_file), index=False)
145     return m

```

B.5 Backtest engine (src/dispersion/backtest/)

Listing 11: src/dispersion/backtest/marking.py

```

1  """
2  Mark aged option positions day by day: interpolate ATM sigma across pillars,
3  look up rates, and rebase strikes for splits.
4
5  from dispersion.backtest.marking import interp_sigma, RateCurve, adjust_strike
6  """
7  import numpy as np
8  import pandas as pd
9
10 PILLARS = (30, 60, 91)
11
12
13 def interp_sigma(sig30, sig60, sig91, tau_days):
14     """
15     ATM sigma at residual maturity 'tau_days', interpolated linearly in total
16     variance  $w = \sigma^2 * \tau$ . Below 30 days we hold sigma(30) (short-end slope
17     dropped, level still tracked). Scalars or aligned arrays; NaN pillars propagate.
18     """
19     sig30 = np.asarray(sig30, dtype=float)

```

```

20 sig60 = np.asarray(sig60, dtype=float)
21 sig91 = np.asarray(sig91, dtype=float)
22 tau = np.asarray(tau_days, dtype=float)
23
24 w30, w60, w91 = sig30**2 * 30.0, sig60**2 * 60.0, sig91**2 * 91.0
25
26 w_lo = w30 + (w60 - w30) * (tau - 30.0) / 30.0      # [30, 60]
27 w_hi = w60 + (w91 - w60) * (tau - 60.0) / 31.0      # [60, 91]
28
29 with np.errstate(invalid="ignore", divide="ignore"):
30     sig_lo = np.sqrt(w_lo / tau)
31     sig_hi = np.sqrt(w_hi / tau)
32
33 out = np.where(
34     tau <= 30.0, sig30,
35     np.where(tau <= 60.0, sig_lo, np.where(tau <= 91.0, sig_hi, sig91)),
36 )
37 return out if out.ndim else float(out)
38
39
40 class RateCurve:
41     """
42     Continuously-compounded zero rate r(date, tau) from the zerocd panel.
43     Linear across the maturity grid, clamped at the ends. If 'date' is missing
44     from zerocd, fall back to the most recent curve within 'max_stale_days'
45     calendar days; beyond that, raise rather than use a stale rate. Parquet is in
46     percent, returned as decimals.
47     """
48
49     def __init__(self, rates: pd.DataFrame, max_stale_days: int = 7):
50         self._dates = pd.DatetimeIndex(np.sort(rates["date"].unique()))
51         self._by_date = {
52             d: (g["days"].to_numpy(dtype=float), g["rate"].to_numpy(dtype=float))
53             for d, g in rates.sort_values("days").groupby("date")
54         }
55         self.max_stale_days = int(max_stale_days)
56
57     def rate(self, date, tau_days: float) -> float:
58         date = pd.Timestamp(date)
59         if date in self._by_date:
60             use = date
61         else:
62             pos = self._dates.searchsorted(date, side="right") - 1
63             if pos < 0:
64                 raise KeyError(f"no zerocd curve on or before {date.date()}")
65             use = self._dates[pos]
66             if (date - use).days > self.max_stale_days:
67                 raise KeyError(f"zerocd gap too wide at {date.date()} (last curve {use.
68 date()}))")
69             days, rate = self._by_date[use]
70             return float(np.interp(float(tau_days), days, rate)) / 100.0

```

```

70
71
72 def adjust_strike(k_entry: float, cfadj_entry: float, cfadj_now: float) -> float:
73     """
74     Strike rebased to today's price scale after splits:  $K_t = K_{\text{entry}} * \text{cfadj\_entry} / \text{cfadj\_t}$ . Quantity scales by the inverse, so a split leaves the
75     invested wealth unchanged.
76     """
77
78     return float(k_entry) * float(cfadj_entry) / float(cfadj_now)

```

Listing 12: src/dispersion/backtest/engine.py

```

1  """
2  Dispersion backtest: short the SPX straddle, long a vega-matched basket of
3  component straddles, delta-hedge the index daily. v0 trades every quarter; v1
4  gates on the signal.
5
6      from dispersion.backtest.engine import run_backtest
7      res = run_backtest()                # v0 (unconditional)
8      res = run_backtest(threshold=0.05)   # v1 (trade only if signal > threshold)
9      res["daily"], res["quarterly"]
10 """
11 import os
12
13 import numpy as np
14 import pandas as pd
15 from scipy.optimize import brentq
16
17 from dispersion.backtest.marking import RateCurve, adjust_strike, interp_sigma
18 from dispersion.utils.greeks import bs_greeks, bs_price
19
20 SPX_SECID = 108105
21 PILLARS = (30, 60, 91)
22 ACT = 365.0
23
24 # Quoted relative bid-ask spreads (% of premium), linear between the 1996 and
25 # 2024 anchors, clamped outside. Entry pays half per leg (impl_premium is
26 # mid-like); the daily index hedge pays 'hedge_bps' of notional; settlement is free.
27 COST_ANCHORS = {"spx": (0.010, 0.006),      # SPX options
28                 "large": (0.070, 0.012),    # constituents ranked 1-50
29                 "small": (0.100, 0.030)}    # constituents ranked 51-100
30 DEFAULT_COSTS = {"hedge_bps": 1e-4, "scale": 1.0}
31
32
33 def parametric_spread(year: int, group: str) -> float:
34     """Quoted relative spread for 'group' in 'year' (linear between anchors)."""
35     v0, v1 = COST_ANCHORS[group]
36     t = min(max(year, 1996), 2024)
37     return v0 + (v1 - v0) * (t - 1996) / (2024 - 1996)
38
39

```

```

40 # ----- #
41 # Entry helpers
42 # ----- #
43 def implied_q(S, k_call, k_put, sig_call, sig_put, T, r, c_mkt, p_mkt,
44               lo=-0.10, hi=0.25) -> float:
45     """
46     Dividend yield backed out of the vendor premiums via C - P (monotone in q).
47     Falls back to 0.0 if there's no root in [lo, hi].
48     """
49     def cp_diff(q):
50         c = bs_price(S, k_call, sig_call, T, r, q, "C")
51         p = bs_price(S, k_put, sig_put, T, r, q, "P")
52         return (c - p) - (c_mkt - p_mkt)
53
54     try:
55         return float(brentq(cp_diff, lo, hi))
56     except ValueError:
57         return 0.0
58
59
60 def exante_quantile_threshold(signal: pd.DataFrame, rebalances, q: float = 0.5,
61                               warmup: int = 12) -> pd.Series:
62     """
63     Per-rebalance ex-ante threshold for v1: quantile 'q' of the daily signal
64     history up to that date (expanding window, so no look-ahead). The first
65     'warmup' rebalances get NaN, which the engine reads as "trade every quarter".
66     """
67     s = signal.copy()
68     s["date"] = pd.to_datetime(s["date"])
69     s = s.set_index("date")["signal"].astype(float).sort_index()
70     out = {}
71     for i, reb in enumerate(rebalances):
72         reb = pd.Timestamp(reb)
73         hist = s.loc[:reb].dropna()
74         out[reb] = np.nan if (i < warmup or hist.empty) else float(hist.quantile(q))
75     return pd.Series(out)
76
77
78 def _entry_leg(rows, cp):
79     r = rows[rows["cp_flag"] == cp]
80     if len(r) != 1:
81         return None
82     r = r.iloc[0]
83     return float(r["iv"]), float(r["premium"]), float(r["strike"])
84
85
86 class _Position:
87     """One straddle: fixed strikes, fixed entry q-hat, split-aware."""
88
89     __slots__ = ("secid", "qty0", "cfadj0", "k_call", "k_put", "q_hat",
90                  "last_mark", "last_greeks", "frozen_days")

```

```

91
92 def __init__(self, secid, qty0, cfadj0, k_call, k_put, q_hat, entry_price):
93     self.secid = secid
94     self.qty0 = qty0
95     self.cfadj0 = cfadj0
96     self.k_call = k_call
97     self.k_put = k_put
98     self.q_hat = q_hat
99     self.last_mark = entry_price          # per-unit straddle value
100     self.last_greeks = {"delta_S": 0.0, "vega_c": 0.0, "vega_p": 0.0,
101                          "gamma": 0.0, "theta": 0.0, "S": np.nan,
102                          "sig_c": np.nan, "sig_p": np.nan}
103     self.frozen_days = 0
104
105 def qty(self, cfadj_now):
106     return self.qty0 * cfadj_now / self.cfadj0
107
108 def strikes(self, cfadj_now):
109     return (adjust_strike(self.k_call, self.cfadj0, cfadj_now),
110            adjust_strike(self.k_put, self.cfadj0, cfadj_now))
111
112
113 # ----- #
114 # Main
115 # ----- #
116 def run_backtest(
117     processed_dir: str = "data/processed",
118     data: dict | None = None,
119     threshold: float | None = None,
120     costs: dict | None = None,
121     cash_tenor: float = 10.0,
122     min_names: int = 80,
123     n_leg: int | None = None,
124 ) -> dict:
125     """
126     Run the backtest over every rebalance.
127
128     threshold=None -> v0 (always trade); a float or per-rebalance pd.Series gates
129     the quarters (v1). costs=None -> gross; costs={} or DEFAULT_COSTS -> net of the
130     parametric spreads (plus hedge_bps on hedge notional; optional 'scale'). 'data'
131     injects the input frames directly for tests, otherwise they're read from parquet.
132     Returns dict(daily, quarterly, ledger).
133     """
134     if data is None:
135         data = {k: pd.read_parquet(os.path.join(processed_dir, f"{k}.parquet"))
136                for k in ("surface", "spots", "rates", "weights", "signal")}
137     surface, spots, rates = data["surface"], data["spots"], data["rates"]
138     weights, signal = data["weights"], data["signal"]
139
140     for df in (surface, spots, signal):
141         df["date"] = pd.to_datetime(df["date"])

```



```

142     for df in (surface, spots, weights):
143         df["rebalance_date"] = pd.to_datetime(df["rebalance_date"])
144
145     rc = RateCurve(rates)
146     sig_at = signal.set_index("date")["signal"]
147     rebals = sorted(weights["rebalance_date"].unique())
148
149     use_costs = costs is not None
150     cost_scale = (costs or {}).get("scale", DEFAULT_COSTS["scale"])
151     hedge_bps = (costs or {}).get("hedge_bps", DEFAULT_COSTS["hedge_bps"]) if
152     use_costs else 0.0
153
154     W = 1.0
155     prev_nav = W
156     daily_rows, q_rows, ledger_rows = [], [], []
157
158     for k, reb in enumerate(rebals[:-1]):
159         reb = pd.Timestamp(reb)
160         nxt = pd.Timestamp(rebals[k + 1])
161
162         surf_q = surface[surface["rebalance_date"] == reb]
163         spot_q = spots[spots["rebalance_date"] == reb]
164         w_q = weights[weights["rebalance_date"] == reb]
165
166         # quarter pivots: sigma pillars per (secid, cp, days); spot close/cfadj
167         piv_iv = surf_q.pivot_table(index="date", columns=["secid", "cp_flag", "days"],
168                                     values="iv", aggfunc="first").astype("float64")
169         piv_close = spot_q.pivot_table(index="date", columns="secid", values="close",
170                                       aggfunc="first").astype("float64")
171         piv_cfadj = spot_q.pivot_table(index="date", columns="secid", values="cfadj",
172                                       aggfunc="first").astype("float64")
173         # force float64 -- nullable dtypes leak pd.NA and break np.isfinite
174         dates = piv_close.index.sort_values()
175
176         # gate: None -> v0; float -> fixed threshold; Series -> per-rebalance
177         # threshold (NaN = warm-up -> trade)
178         if threshold is None:
179             traded = True
180         else:
181             thr = float(threshold.get(reb, np.nan)) if isinstance(threshold, pd.Series)
182             \
183                 else float(threshold)
184             if np.isnan(thr):
185                 traded = True
186             else:
187                 s_val = float(sig_at.loc[reb]) if reb in sig_at.index else np.nan
188                 traded = bool(np.isfinite(s_val) and s_val > thr)
189
190         # ----- #
191         # Entry at the rebalance close

```

```

190 # ----- #
191 book, cash, n_hedge = [], W, 0.0
192 y_sum = lam = np.nan
193 held_n = 0
194 c_entry_q = c_hedge_q = 0.0
195 if traded:
196     T0 = (nxt - reb).days / ACT
197     r0 = rc.rate(reb, (nxt - reb).days)
198     e = surf_q[(surf_q["date"] == reb) & (surf_q["days"] == 91)]
199
200     entries = {}
201     for secid, rows in e.groupby("secid"):
202         c = _entry_leg(rows, "C")
203         p = _entry_leg(rows, "P")
204         S0 = piv_close.at[reb, secid] if secid in piv_close.columns else np.
nan
205         if c is None or p is None or not np.isfinite(S0):
206             continue
207         sc, prem_c, kc = c
208         sp_, prem_p, kp = p
209         qh = implied_q(S0, kc, kp, sc, sp_, T0, r0, prem_c, prem_p)
210         g_c = bs_greeks(S0, kc, sc, T0, r0, qh, "C")
211         g_p = bs_greeks(S0, kp, sp_, T0, r0, qh, "P")
212         price = prem_c + prem_p
213         entries[secid] = dict(
214             price=price, kc=kc, kp=kp, qh=qh,
215             nu=(g_c["vega"] + g_p["vega"]) / price, # relative vega
216             sig_bar=0.5 * (sc + sp_),
217             cfadj0=float(piv_cfadj.at[reb, secid]),
218         )
219
220     if SPX_SECID not in entries:
221         raise ValueError(f"no SPX entry at {reb.date()}")
222     idx_e = entries.pop(SPX_SECID)
223
224     w_map = w_q.dropna(subset=["secid"]).set_index("secid")["weight"]
225     held = [s for s in entries if s in w_map.index]
226     # only trade the top n_leg names by cap weight; w_til renormalises over
227     # them so the leg stays vega-neutral. Fewer, larger legs -> fewer
228     # spreads, and tight large-cap spreads replace the wide small-cap tail.
229     if n_leg is not None and len(held) > n_leg:
230         held = w_map.loc[held].sort_values(ascending=False).head(n_leg).index.
tolist()
231     held_n = len(held)
232     floor = min(n_leg, 10) if n_leg is not None else min_names
233     if held_n < floor:
234         raise ValueError(f"only {held_n} names with entry premiums at {reb.
date()}")
235
236     w_til = w_map.loc[held] / w_map.loc[held].sum()
237     denom = sum(w_til[s] * entries[s]["nu"] * entries[s]["sig_bar"] for s in

```

```

held)
238     lam = idx_e["nu"] * idx_e["sig_bar"] / denom        # vega-neutral to a
proportional shock
239     y = {s: lam * w_til[s] for s in held}
240     y_sum = float(sum(y.values()))
241
242     # index short: -100% of wealth; components: +y_i of wealth
243     book.append(_Position(SPX_SECID, -W / idx_e["price"], idx_e["cfadj0"],
244                          idx_e["kc"], idx_e["kp"], idx_e["qh"], idx_e["price"
]))
245     cash += W                                           # short proceeds
246     for s in held:
247         q_i = y[s] * W / entries[s]["price"]
248         book.append(_Position(s, q_i, entries[s]["cfadj0"], entries[s]["kc"],
249                             entries[s]["kp"], entries[s]["qh"], entries[s]["
price"])))
250         cash -= y[s] * W
251
252     if use_costs: # pay half the quoted spread on each entry leg
253         rnk_map = w_q.dropna(subset=["secid"]).set_index("secid")["rnk"] \
254             if "rnk" in w_q.columns else pd.Series(dtype=float)
255         c_entry_q = 0.5 * cost_scale * parametric_spread(reb.year, "spx") * W
256         for s in held:
257             grp = "large" if float(rnk_map.get(s, 1)) <= 50 else "small"
258             c_entry_q += 0.5 * cost_scale * parametric_spread(reb.year, grp) *
y[s] * W
259         cash -= c_entry_q
260
261     # ----- #
262     # Daily loop: mark, delta-hedge with the index, accrue cash
263     # ----- #
264     def _mark(pos, t, tau_days):
265         """Per-unit straddle mark + greeks; freezes on missing data."""
266         secid = pos.secid
267         S = piv_close.at[t, secid] if (secid in piv_close.columns
268                                     and t in piv_close.index) else np.nan
269         cf = piv_cfadj.at[t, secid] if np.isfinite(S) else np.nan
270         sigs = {}
271         for cp in ("C", "P"):
272             try:
273                 s30 = piv_iv.at[t, (secid, cp, 30)]
274                 s60 = piv_iv.at[t, (secid, cp, 60)]
275                 s91 = piv_iv.at[t, (secid, cp, 91)]
276             except KeyError:
277                 s30 = s60 = s91 = np.nan
278             sigs[cp] = interp_sigma(s30, s60, s91, tau_days)
279         if not (np.isfinite(S) and np.isfinite(sigs["C"]) and np.isfinite(sigs["P"
]))
280             and np.isfinite(cf)):
281             pos.frozen_days += 1
282             return pos.last_mark, pos.last_greeks, None

```

```

283     kc, kp = pos.strikes(cf)
284     T = max(tau_days, 0.5) / ACT
285     r = rc.rate(t, max(tau_days, 1.0))
286     g_c = bs_greeks(S, kc, sigs["C"], T, r, pos.q_hat, "C")
287     g_p = bs_greeks(S, kp, sigs["P"], T, r, pos.q_hat, "P")
288     mark = g_c["price"] + g_p["price"]
289     greeks = {"delta_S": (g_c["delta"] + g_p["delta"]) * S,
290              "vega_c": g_c["vega"], "vega_p": g_p["vega"],
291              "gamma": g_c["gamma"] + g_p["gamma"],
292              "theta": g_c["theta"] + g_p["theta"],
293              "S": S, "sig_c": sigs["C"], "sig_p": sigs["P"]}
294     pos.last_mark, pos.last_greeks = mark, greeks
295     return mark, greeks, cf
296
297 def _book_state(t, tau_days):
298     st = {"nav_pos": 0.0, "delta_dollar": 0.0, "nav_idx": 0.0, "nav_comp": 0.0,
299
300           "theta": 0.0, "vega": 0.0, "gamma_S2": 0.0}
301     for pos in book:
302         mark, greeks, cf = _mark(pos, t, tau_days)
303         qty = pos.qty(cf) if cf is not None else pos.qty(pos.cfadj0)
304         v = qty * mark
305         st["nav_pos"] += v
306         st["delta_dollar"] += qty * greeks["delta_S"]
307         st["theta"] += qty * greeks["theta"]
308         st["vega"] += qty * (greeks["vega_c"] + greeks["vega_p"])
309         if np.isfinite(greeks["S"]):
310             st["gamma_S2"] += qty * greeks["gamma"] * greeks["S"] ** 2
311         if pos.secid == SPX_SECID:
312             st["nav_idx"] += v
313         else:
314             st["nav_comp"] += v
315     return st
316
317 # entry-day state (hedge set at the entry close)
318 spx_S = lambda t: piv_close.at[t, SPX_SECID]
319 if traded:
320     st0 = _book_state(reb, (nxt - reb).days)
321     n_hedge = -st0["delta_dollar"] / spx_S(reb)
322     cash -= n_hedge * spx_S(reb)
323     c = hedge_bps * abs(n_hedge) * spx_S(reb)
324     cash -= c
325     c_hedge_q += c
326
327 prev_t = reb
328 for t in dates[dates > reb]:
329     dt_cal = (t - prev_t).days
330     cash *= float(np.exp(rc.rate(prev_t, cash_tenor) * dt_cal / ACT))
331     tau = (nxt - t).days
332     if traded:
333         st = _book_state(t, tau)

```

```

333         nav = cash + st["nav_pos"] + n_hedge * spx_S(t)
334         ledger_rows.append((t, reb, nav, cash, st["nav_idx"], st["nav_comp"],
335                             n_hedge, float(spx_S(t)), st["theta"], st["vega"],
336                             st["gamma_S2"], st["delta_dollar"]))
337         # re-hedge at the close
338         n_new = -st["delta_dollar"] / spx_S(t)
339         cash -= (n_new - n_hedge) * spx_S(t)
340         c = hedge_bps * abs(n_new - n_hedge) * spx_S(t)
341         cash -= c
342         c_hedge_q += c
343         n_hedge = n_new
344     else:
345         nav = cash
346         daily_rows.append((t, reb, nav / prev_nav - 1.0, nav, held_n, traded))
347         prev_nav = nav
348         prev_t = t
349
350     # ----- #
351     # Settlement at the next rebalance (intrinsic payoff)
352     # ----- #
353     dt_cal = (nxt - prev_t).days
354     cash *= float(np.exp(rc.rate(prev_t, cash_tenor) * dt_cal / ACT))
355     n_frozen_settle = 0
356     if traded:
357         spot_n = spots[(spots["rebalance_date"] == nxt) & (spots["date"] == nxt)]
358         s_map = spot_n.set_index("secid")["close"]
359         cf_map = spot_n.set_index("secid")["cfadj"]
360         for pos in book:
361             if pos.secid in s_map.index:
362                 S, cf = float(s_map.loc[pos.secid]), float(cf_map.loc[pos.secid])
363             else: # left the universe / delisted: last frozen mark's spot
364                 S, cf = pos.last_greeks["S"], pos.cfadj0
365                 n_frozen_settle += 1
366                 if not np.isfinite(S):
367                     S, cf = 0.0, pos.cfadj0 # worthless fallback, counted
368                 kc, kp = pos.strikes(cf)
369                 payoff = max(S - kc, 0.0) + max(kp - S, 0.0)
370                 cash += pos.qty(cf) * payoff
371             s_settle = float(s_map.loc[SPX_SECID])
372             cash += n_hedge * s_settle
373             c = hedge_bps * abs(n_hedge) * s_settle
374             cash -= c
375             c_hedge_q += c
376             n_hedge = 0.0
377
378     nav = cash
379     daily_rows.append((nxt, reb, nav / prev_nav - 1.0, nav, held_n, traded))
380     prev_nav = nav
381
382     q_rows.append((reb, nxt, traded, held_n, nav / W - 1.0, y_sum, lam,
383                   float(np.mean([p.frozen_days for p in book])) if book else 0.0,

```

```

384         n_frozen_settle, c_entry_q / W, c_hedge_q / W))
385     W = nav
386
387     daily = pd.DataFrame(daily_rows,
388                         columns=["date", "rebalance_date", "ret", "nav",
389                                "n_names", "traded"])
390     quarterly = pd.DataFrame(q_rows,
391                             columns=["rebalance_date", "settle_date", "traded",
392                                    "n_names", "ret_q", "y_sum", "lam",
393                                    "avg_frozen_days", "n_frozen_settle",
394                                    "cost_entry", "cost_hedge"])
395     ledger = pd.DataFrame(ledger_rows,
396                          columns=["date", "rebalance_date", "nav", "cash", "nav_idx",
397                                 "nav_comp", "hedge_shares", "spx_close", "theta",
398                                 "vega", "gamma_S2", "delta_dollar"])
399     return {"daily": daily, "quarterly": quarterly, "ledger": ledger}

```

B.6 Machine learning layer (src/dispersion/ml/)

Listing 13: src/dispersion/ml/features.py

```

1  """
2  Build the ML feature table from the existing parquets (no new WRDS pull):
3  spectral features from the cleaned correlation matrix, SPX vol structure,
4  realised vol / VRP, the component-IV cross-section, and correlation levels. The
5  label y_spike is future minus current realised correlation, used in training only.
6
7      from dispersion.ml.features import build_ml_dataset
8      df = build_ml_dataset()          # writes data/processed/ml_dataset.parquet
9  """
10 import os
11
12 import numpy as np
13 import pandas as pd
14
15 from ..backtest.engine import SPX_SECID, parametric_spread
16
17 HORIZON = 63          # trading days ~ the 91-calendar-day decision horizon
18 D21 = 21              # short dynamics window
19
20 # core set for the Gaussian models -- a full-cov fit in 32 dims wouldn't survive
21 # ~2,000 walk-forward observations
22 CORE_SET = ["f_lam1", "f_dlam1_21", "f_turb21", "f_vrp", "f_term_slope",
23            "f_anchor_gap", "f_iv_spx", "f_rot21"]
24
25
26 def build_ml_dataset(
27     processed_dir: str = "data/processed",
28     out_file: str | None = "ml_dataset.parquet",
29 ) -> pd.DataFrame:
30     sig = pd.read_parquet(os.path.join(processed_dir, "signal.parquet"))

```

```

31 sigr = pd.read_parquet(os.path.join(processed_dir, "signal_rmt.parquet"))
32 rmt = pd.read_parquet(os.path.join(processed_dir, "rmt_daily.parquet"))
33 ivx = pd.read_parquet(os.path.join(processed_dir, "iv_index.parquet"))
34 surface = pd.read_parquet(os.path.join(processed_dir, "surface.parquet"))
35 spots = pd.read_parquet(os.path.join(processed_dir, "spots.parquet"))
36 ivc = pd.read_parquet(os.path.join(processed_dir, "iv_components.parquet"))
37 wts = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
38 vix = pd.read_parquet(os.path.join(processed_dir, "vix.parquet")) # CBOE (cboe.
cboe)
39 for df in (sig, sigr, rmt, ivx, surface, spots, ivc, vix):
40     df["date"] = pd.to_datetime(df["date"])
41
42 # SPX term slope (30d -> 91d ATM pillars, C/P averaged)
43 spx_surf = surface[(surface["secid"] == SPX_SECID) & (surface["days"].isin([30,
91]))]
44 piv = (spx_surf.groupby(["date", "days"])["iv"].mean().unstack("days")
45         .astype("float64"))
46 term = ((piv[91] - piv[30]) / piv[30]).rename("f_term_slope")
47
48 # SPX realised vol block (vendor closes, same source as the strikes)
49 spx_close = (spots[spots["secid"] == SPX_SECID]
50              .drop_duplicates("date").set_index("date")["close"]
51              .astype("float64").sort_index())
52 lr = np.log(spx_close).diff()
53 rv21 = (lr.rolling(D21, min_periods=15).std() * np.sqrt(252.0)).rename("f_rv21")
54 rv63 = (lr.rolling(63, min_periods=50).std() * np.sqrt(252.0)).rename("f_rv63")
55 dd252 = (spx_close / spx_close.rolling(252, min_periods=100).max() - 1.0).rename("
f_dd252")
56 mom63 = (spx_close.pct_change(63)).rename("f_mom63")
57
58 # component-IV cross-section (weighted, renormalised)
59 ivw = ivc.merge(wts[["rebalance_date", "permno", "weight"]],
60                on=["rebalance_date", "permno"], how="left")
61 ivw = ivw.dropna(subset=["iv_atm"])
62 g = ivw.assign(ws=ivw["weight"] * ivw["iv_atm"]).groupby("date")
63 iv_comp = (g["ws"].sum() / g["weight"].sum()).astype("float64").rename("f_iv_comp"
)
64 iv_xdisp = g["iv_atm"].std().astype("float64").rename("f_iv_xdisp")
65
66 # assemble everything on the date spine
67 m = (
68     sig[["date", "rho_implied", "rho_trailing", "signal"]]
69     .rename(columns={"rho_trailing": "rho_trail63", "signal": "f_sig_base"})
70     .merge(sigr[["date", "signal"]].rename(columns={"signal": "f_sig_rmt"}),
71           on="date", validate="1:1")
72     .merge(rmt[["date", "rho_rmt_clean", "lam1_share", "k_signal", "absorption_top
",
73              "rotation", "lam2_share", "spec_entropy", "pr_v1", "corr_xdisp",
74              "turb"]], on="date", validate="1:1")
75     .merge(ivx[["date", "iv_atm", "iv_call_50", "iv_put_50"]], on="date",
76           how="left", validate="1:1")

```

```

77     .sort_values("date").reset_index(drop=True)
78 )
79 for s in (term, rv21, rv63, dd252, mom63, iv_comp, iv_xdisp):
80     m = m.merge(s.reset_index(), on="date", how="left")
81 m = m.merge(vix, on="date", how="left", validate="1:1")
82
83 out = pd.DataFrame({"date": m["date"]})
84 # A. spectral
85 out["f_lam1"] = m["lam1_share"]
86 out["f_lam2"] = m["lam2_share"]
87 out["f_k"] = m["k_signal"]
88 out["f_abs"] = m["absorption_top"]
89 out["f_entropy"] = m["spec_entropy"]
90 out["f_prv1"] = m["pr_v1"]
91 out["f_xdisp"] = m["corr_xdisp"]
92 out["f_dlam1_21"] = m["lam1_share"].diff(D21)
93 out["f_dlam1_63"] = m["lam1_share"].diff(63)
94 out["f_rot21"] = m["rotation"].rolling(D21, min_periods=10).mean()
95 out["f_turb"] = m["turb"]
96 out["f_turb21"] = m["turb"].rolling(D21, min_periods=10).mean()
97 # B. vol structure
98 out["f_iv_spx"] = m["iv_atm"]
99 out["f_div_21"] = m["iv_atm"].diff(D21)
100 out["f_vix"] = m["vix"] / 100.0 # VIX in decimal units
101 out["f_term_slope"] = m["f_term_slope"]
102 out["f_skew50"] = m["iv_put_50"] - m["iv_call_50"]
103 out["f_volofvol"] = m["iv_atm"].rolling(D21, min_periods=15).std()
104 # C. realised & VRP
105 out["f_rv21"] = m["f_rv21"]
106 out["f_rv63"] = m["f_rv63"]
107 out["f_vrp"] = m["iv_atm"] - m["f_rv63"]
108 out["f_dd252"] = m["f_dd252"]
109 out["f_mom63"] = m["f_mom63"]
110 # D. IV cross-section
111 out["f_iv_comp"] = m["f_iv_comp"]
112 out["f_iv_xdisp"] = m["f_iv_xdisp"]
113 # E. correlation levels / signal / cost
114 out["f_rho_imp"] = m["rho_implied"]
115 out["f_drho_imp_21"] = m["rho_implied"].diff(D21) # the market starting to price
# stress
116 out["f_rho_trail63"] = m["rho_trail63"]
117 out["f_rho_clean252"] = m["rho_rmt_clean"]
118 out["f_drho_21"] = m["rho_trail63"].diff(D21)
119 out["f_sig_base"] = m["f_sig_base"]
120 out["f_sig_rmt"] = m["f_sig_rmt"]
121 out["f_anchor_gap"] = m["rho_trail63"] - m["rho_rmt_clean"]
122 out["f_sig_rmt_pct"] = (m["f_sig_rmt"].expanding(min_periods=252)
123                      .rank(pct=True)) # expanding percentile, no look-
ahead
124 year = m["date"].dt.year
125 out["f_cost_era"] = [

```



```

126         0.5 * (parametric_spread(y, "spx")
127             + 0.7 * parametric_spread(y, "large") + 0.3 * parametric_spread(y, "
small"))
128     for y in year
129 ]
130 # label uses the future, so it's training-only and purged in every fit
131 out["y_spike"] = m["rho_trail63"].shift(-HORIZON) - m["rho_trail63"]
132
133 # force float64 everywhere -- nullable dtypes break numpy/sklearn
134 num_cols = [c for c in out.columns if c != "date"]
135 out[num_cols] = out[num_cols].astype("float64")
136
137 if out_file:
138     out.to_parquet(os.path.join(processed_dir, out_file), index=False)
139 return out

```

Listing 14: src/dispersion/ml/regime.py

```

1  """
2  Daily danger-regime probability from a GMM and a filtered HMM on the core stress
3  features, both refit walk-forward at each rebalance. The danger state is named
4  from the training window only, and the HMM uses the filtered (forward-pass)
5  posterior rather than the smoothed one, so nothing looks ahead. The feature is
6  the mean of the two probabilities. Causality is checked in tests/test_regime.py.
7
8      from dispersion.ml.regime import build_regime_feature
9      build_regime_feature()      # adds regime probs to ml_dataset.parquet
10 """
11 import os
12 import warnings
13
14 import numpy as np
15 import pandas as pd
16 from hmmlearn.hmm import GaussianHMM
17 from scipy.special import logsumexp
18 from sklearn.mixture import GaussianMixture
19
20 CORE = ["f_lam1", "f_dlam1_63", "f_turb21", "f_vix"]
21 # every core feature is oriented "higher = more stress", so we name the danger
22 # state by the highest composite centroid instead of any single axis (turbulence
23 # alone doesn't work -- EWMA devol flattens it)
24 WARMUP = 756          # ~3y of complete core rows before the first fit
25 N_STATES = 3
26
27
28 def hmm_filtered_posterior(model: GaussianHMM, X: np.ndarray) -> np.ndarray:
29     """
30     Filtered posterior P(state_t | obs_1..t) via a forward pass -- at each t it
31     only uses observations up to t, never later ones. Returns (T, K).
32     """
33     logB = model._compute_log_likelihood(X)          # (T, K) log emission probs

```

```

34 log_pi = np.log(model.startprob_ + 1e-300)
35 log_A = np.log(model.transmat_ + 1e-300)
36 T, K = logB.shape
37 log_alpha = np.empty((T, K))
38 log_alpha[0] = log_pi + logB[0]
39 log_alpha[0] -= logsumexp(log_alpha[0])           # normalise (filtered)
40 for t in range(1, T):
41     log_alpha[t] = logsumexp(log_alpha[t - 1][:, None] + log_A, axis=0) + logB[t]
42     log_alpha[t] -= logsumexp(log_alpha[t])
43 return np.exp(log_alpha)
44
45
46 def _danger_index(centroids: np.ndarray) -> int:
47     """
48     Danger = the component/state whose centroid has the highest mean across the
49     core features. Centroids are already in training z-score units, so a plain
50     mean is comparable across features.
51     """
52     return int(np.argmax(centroids.mean(axis=1)))
53
54
55 def regime_probs(feat: pd.DataFrame, rebals, core=CORE, warmup: int = WARMUP,
56                 seed: int = 0):
57     """
58     Walk-forward danger probabilities for a core feature subset. 'feat' is
59     date-indexed with at least the 'core' columns. Returns (p_gmm, p_hmm) aligned
60     to feat.index -- used for both the full-core feature and the VIX-only variant.
61     """
62     feat = feat[core]
63     p_gmm = pd.Series(np.nan, index=feat.index)
64     p_hmm = pd.Series(np.nan, index=feat.index)
65
66     for k, reb in enumerate(rebals):
67         reb = pd.Timestamp(reb)
68         nxt = pd.Timestamp(rebals[k + 1]) if k + 1 < len(rebals) \
69             else feat.index[-1] + pd.Timedelta(days=1)
70
71         train = feat.loc[:reb].dropna()
72         if len(train) < warmup:
73             continue
74
75         mu, sd = train.mean(), train.std().replace(0, 1.0)
76         Ztr = ((train - mu) / sd).to_numpy(dtype="float64")
77
78         gmm = GaussianMixture(N_STATES, covariance_type="full", random_state=seed,
79                               reg_covar=1e-4).fit(Ztr)
80         gd = _danger_index(gmm.means_)
81         hmm = GaussianHMM(N_STATES, covariance_type="full", random_state=seed,
82                           n_iter=100, tol=1e-3)
83         hmm.fit(Ztr)
84         hd = _danger_index(hmm.means_)

```

```

85
86     seg = feat.loc[reb:nxt].iloc[:-1] if nxt in feat.index else feat.loc[reb:nxt]
87     seg = seg.dropna()
88     if seg.empty:
89         continue
90     Zseg = ((seg - mu) / sd).to_numpy(dtype="float64")
91     p_gmm.loc[seg.index] = gmm.predict_proba(Zseg)[: , gd]
92
93     hist = feat.loc[:reb].dropna()
94     combo = pd.concat([hist, seg[~seg.index.isin(hist.index)]]))
95     Zc = ((combo - mu) / sd).to_numpy(dtype="float64")
96     post = hmm_filtered_posterior(hmm, Zc)
97     p_hmm.loc[seg.index] = pd.Series(post[:, hd], index=combo.index).loc[seg.index]
98     ].to_numpy()
99
100     return p_gmm, p_hmm
101
102 def build_regime_feature(
103     processed_dir: str = "data/processed",
104     core=CORE,
105     warmup: int = WARMUP,
106     out_file: str | None = "ml_dataset.parquet",
107     seed: int = 0,
108 ) -> pd.DataFrame:
109     """Walk-forward causal danger-regime probabilities, merged into ml_dataset."""
110     ml = pd.read_parquet(os.path.join(processed_dir, "ml_dataset.parquet"))
111     weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
112     ml["date"] = pd.to_datetime(ml["date"])
113     weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
114
115     feat = ml[["date"] + core].set_index("date")
116     rebals = sorted(weights["rebalance_date"].unique())
117     p_gmm, p_hmm = regime_probs(feat, rebals, core=core, warmup=warmup, seed=seed)
118
119     out = ml.copy()
120     out["f_reg_gmm"] = p_gmm.to_numpy()
121     out["f_reg_hmm"] = p_hmm.to_numpy()
122     with warnings.catch_warnings():
123         # warm-up rows are NaN in both models -> nanmean warns on those; ignore it
124         warnings.simplefilter("ignore", RuntimeWarning)
125         out["f_regime"] = np.nanmean(np.c_[p_gmm.to_numpy(), p_hmm.to_numpy()], axis
126 =1)
127     num = [c for c in out.columns if c != "date"]
128     out[num] = out[num].astype("float64")
129
130     if out_file:
131         out.to_parquet(os.path.join(processed_dir, out_file), index=False)
132     return out

```

Listing 15: src/dispersion/ml/metamodel.py

```

1  """
2  Predict each quarter's trade return, walk-forward and purged, then veto the
3  trades in the bad tail. Sits on top of the vl_rmt gate. Sample is small
4  (~90 quarters), so read the results with that in mind.
5
6  Usage:
7      from dispersion.ml.metamodel import run_central_test
8      res = run_central_test()
9  """
10 import os
11
12 import numpy as np
13 import pandas as pd
14 from xgboost import XGBRegressor
15
16 from ..backtest.engine import exante_quantile_threshold, run_backtest
17 from .regime import regime_probs
18
19 HORIZON, EMBARGO = 63, 21          # trading days
20 MIN_TRAIN = 24                    # quarters before the first prediction (~6y)
21 VETO_PCTILE = 0.33                # veto the bottom third of predicted returns
22
23 XGB_PARAMS = dict(objective="reg:squarederror", max_depth=2, n_estimators=100,
24                    learning_rate=0.05, subsample=0.8, colsample_bytree=0.8,
25                    min_child_weight=5, reg_lambda=1.0, random_state=0)
26
27 FEATURE_SETS = {
28     "A_vix":      ["f_vix", "f_term_slope", "f_regime_vix"],
29     "B_spectral": ["f_vix", "f_term_slope", "f_regime_vix",
30                    "f_lam1", "f_dlam1_63", "f_turb21", "f_k", "f_rot21", "f_regime"],
31     "Full":       ["f_vix", "f_term_slope", "f_lam1", "f_dlam1_63", "f_turb21",
32                    "f_k", "f_rot21", "f_vrp", "f_drho_imp_21", "f_sig_rmt", "f_regime"],
33 }
34
35
36 def _snapshots(processed_dir):
37     """Features at each rebalance close, joined to the quarter's v0 return as the
38     label."""
39     ml = pd.read_parquet(os.path.join(processed_dir, "ml_dataset.parquet"))
40     ml["date"] = pd.to_datetime(ml["date"])
41     q0 = pd.read_parquet(os.path.join(processed_dir, "backtest_v0_quarterly.parquet"))
42     q0["rebalance_date"] = pd.to_datetime(q0["rebalance_date"])
43
44     # VIX-only regime variant, fit on vol features only so the A feature set stays
45     # pure
46     weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
47     weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
48     rebals = sorted(weights["rebalance_date"].unique())

```

```

47 feat_vix = ml.set_index("date")[["f_vix", "f_term_slope"]]
48 pg, ph = regime_probs(feats_vix, rebals, core=["f_vix", "f_term_slope"])
49 ml["f_regime_vix"] = np.nanmean(np.c_[pg.to_numpy(), ph.to_numpy()], axis=1)
50
51 snap = ml.merge(q0[["rebalance_date", "ret_q"]].rename(
52     columns={"rebalance_date": "date", "ret_q": "y"}), on="date", how="inner")
53 return snap.sort_values("date").reset_index(drop=True)
54
55
56 def walk_forward_predict(snap: pd.DataFrame, features: list) -> pd.Series:
57     """
58     Walk-forward prediction of y at each rebalance. Only trains on quarters whose
59     label horizon plus embargo ends before the test date, so no future leaks in.
60     """
61     dates = snap["date"].to_numpy()
62     yhat = pd.Series(np.nan, index=snap.index)
63     for i in range(len(snap)):
64         test_date = snap["date"].iloc[i]
65         cutoff = test_date - pd.Timedelta(days=int((HORIZON + EMBARGO) * 7 / 5)) #
66         trading days -> calendar
67         train = snap.iloc[:i][snap["date"].iloc[:i] <= cutoff]
68         train = train.dropna(subset=features + ["y"])
69         if len(train) < MIN_TRAIN:
70             continue
71         Xtr, ytr = train[features].to_numpy("float64"), train["y"].to_numpy("float64")
72         xte = snap[features].iloc[[i]]
73         if xte.isna().any(axis=None):
74             continue
75         model = XGBRegressor(**XGB_PARAMS)
76         model.fit(Xtr, ytr)
77         yhat.iloc[i] = float(model.predict(xte.to_numpy("float64"))[0])
78     return yhat
79
80 def _veto_mask(yhat: pd.Series) -> pd.Series:
81     """Veto quarter i if its prediction is below the expanding VETO_PCTILE of past
82     predictions."""
83     veto = pd.Series(False, index=yhat.index)
84     hist = []
85     for i in range(len(yhat)):
86         yi = yhat.iloc[i]
87         if np.isfinite(yi) and len(hist) >= MIN_TRAIN:
88             veto.iloc[i] = yi < np.quantile(hist, VETO_PCTILE)
89         if np.isfinite(yi):
90             hist.append(yi)
91     return veto
92
93 def predictive_metrics(snap, yhat, crisis_q=0.10):
94     """Directional accuracy, plus precision and recall of the veto on crisis quarters.
95     """

```

```

95     m = snap.assign(yhat=yhat.to_numpy()).dropna(subset=["yhat", "y"])
96     if len(m) < MIN_TRAIN:
97         return {}
98     crisis = m["y"] <= m["y"].quantile(crisis_q)           # worst quarters ex-post,
used only to score the veto
99     veto = _veto_mask(pd.Series(m["yhat"].to_numpy(), index=m.index)).to_numpy()
100     tp = int((veto & crisis.to_numpy()).sum())
101     fp = int((veto & ~crisis.to_numpy()).sum())
102     fn = int((~veto & crisis.to_numpy()).sum())
103     return {
104         "n_pred": int(len(m)),
105         "corr_yhat_y": float(np.corrcoef(m["yhat"], m["y"])[0, 1]),
106         "dir_acc": float((np.sign(m["yhat"]) == np.sign(m["y"])).mean()),
107         "n_crisis": int(crisis.sum()),
108         "recall_crisis": tp / (tp + fn) if tp + fn else np.nan,
109         "precision_veto": tp / (tp + fp) if tp + fp else np.nan,
110     }
111
112
113 def feature_importance(snap, features):
114     """Mean XGBoost gain importance across a few expanding folds."""
115     imp = np.zeros(len(features))
116     n = 0
117     for cut in (0.5, 0.7, 0.9):
118         k = int(len(snap) * cut)
119         tr = snap.iloc[:k].dropna(subset=features + ["y"])
120         if len(tr) < MIN_TRAIN:
121             continue
122         m = XGBRegressor(**XGB_PARAMS).fit(tr[features].to_numpy("float64"),
123                                             tr["y"].to_numpy("float64"))
124         imp += m.feature_importances_
125         n += 1
126     imp = imp / max(n, 1)
127     return dict(sorted(zip(features, imp.tolist()), key=lambda kv: -kv[1]))
128
129
130 def run_central_test(processed_dir: str = "data/processed"):
131     """Run predictions, veto, and gross+net backtests for each of the A/B/Full feature
sets."""
132     snap = _snapshots(processed_dir)
133     signal_rmt = pd.read_parquet(os.path.join(processed_dir, "signal_rmt.parquet"))
134     weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
135     signal_rmt["date"] = pd.to_datetime(signal_rmt["date"])
136     weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
137     rebals = sorted(weights["rebalance_date"].unique())
138     base = {k: pd.read_parquet(os.path.join(processed_dir, f"{k}.parquet"))
139             for k in ("surface", "spots", "rates", "weights")}
140     v1rmt_thr = exante_quantile_threshold(signal_rmt, rebals, q=0.5, warmup=12)
141
142     out = {"snap_dates": snap["date"], "y": snap["y"], "arms": {}}
143     for name, feats in FEATURE_SETS.items():

```

```

144     yhat = walk_forward_predict(snap, feats)
145     veto = _veto_mask(yhat)
146     veto_dates = set(snap.loc[veto.to_numpy(), "date"])
147     thr = v1rmt_thr.copy()
148     for d in veto_dates:
149         thr[pd.Timestamp(d)] = 1e9 # unreachable threshold ->
150     skip the quarter
151     res = {}
152     for tag, kw in [("gross", {}), ("net", dict(costs={}))]:
153         bt = run_backtest(data={**{k: v.copy() for k, v in base.items()},
154                                "signal": signal_rmt.copy()}, threshold=thr, **kw)
155         bt["quarterly"].to_parquet(
156             os.path.join(processed_dir, f"backtest_ml_{name}_{tag}_quarterly.
157             parquet"), index=False)
158         bt["daily"].to_parquet(
159             os.path.join(processed_dir, f"backtest_ml_{name}_{tag}_daily.parquet"),
160             index=False)
161         res[tag] = bt["quarterly"]
162         out["arms"][name] = {
163             "yhat": yhat, "veto": veto, "n_veto": int(veto.sum()),
164             "pred": predictive_metrics(snap, yhat),
165             "importance": feature_importance(snap, feats),
166         }
167     return out

```

Listing 16: src/dispersion/ml/experiments.py

```

1  """
2  Two experimental vetoes that sit alongside the main ML, neither overwrites it.
3
4  Lever 1 predicts the forward correlation spike on daily data and vetoes a trade
5  when the spike predicted at the rebalance is in the high tail. Lever 3 skips the
6  model entirely and vetoes on the regime probability instead. Both layer on top
7  of the v1_rmt gate.
8
9  Usage:
10     from dispersion.ml.experiments import run_levers
11     run_levers() # writes backtest_regonly_* and backtest_spike_* + a summary
12 """
13 import json
14 import os
15
16 import numpy as np
17 import pandas as pd
18 from xgboost import XGBRegressor
19
20 from ..backtest.engine import exante_quantile_threshold, run_backtest
21
22 HORIZON, EMBARGO = 63, 21
23 MIN_TRAIN_DAYS = 756
24 VETO_Q = 0.67 # veto the top third of predicted danger

```

```

25
26 SPIKE_FEATS = ["f_vix", "f_term_slope", "f_lam1", "f_dlam1_63", "f_turb21", "f_k",
27               "f_rot21", "f_vrp", "f_drho_imp_21", "f_sig_rmt", "f_regime"]
28 XGB = dict(objective="reg:squarederror", max_depth=3, n_estimators=200,
29            learning_rate=0.05, subsample=0.8, colsample_bytree=0.8,
30            min_child_weight=20, reg_lambda=1.0, random_state=0)
31
32
33 def predict_spike_daily(processed_dir="data/processed", features=SPIKE_FEATS):
34     """
35     Walk-forward daily spike prediction. Refit at each rebalance on the purged
36     expanding history, then predict that quarter's days. Returns the prediction at
37     each rebalance and oos_corr, the correlation between predicted and realised
38     spike over all out-of-sample days (tells you whether the spike is predictable).
39     """
40     ml = pd.read_parquet(os.path.join(processed_dir, "ml_dataset.parquet"))
41     weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
42     ml["date"] = pd.to_datetime(ml["date"])
43     weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
44     rebals = sorted(weights["rebalance_date"].unique())
45     idx = ml.set_index("date")
46
47     purge_cal = int((HORIZON + EMBARGO) * 7 / 5)
48     yhat_reb = {}
49     oos_pred, oos_true = [], []
50     for k, reb in enumerate(rebals):
51         reb = pd.Timestamp(reb)
52         nxt = pd.Timestamp(rebals[k + 1]) if k + 1 < len(rebals) else idx.index[-1] +
pd.Timedelta(days=1)
53         cutoff = reb - pd.Timedelta(days=purge_cal)
54         tr = ml[ml["date"] <= cutoff].dropna(subset=features + ["y_spike"])
55         if len(tr) < MIN_TRAIN_DAYS:
56             continue
57         m = XGBRegressor(**XGB).fit(tr[features].to_numpy("float64"),
58                                   tr["y_spike"].to_numpy("float64"))
59         # value used by the gate at the rebalance
60         row = idx.loc[[reb], features] if reb in idx.index else None
61         if row is not None and row.notna().all(axis=1).iloc[0]:
62             yhat_reb[reb] = float(m.predict(row.to_numpy("float64"))[0])
63         # out-of-sample check on the quarter's days
64         seg = ml[(ml["date"] >= reb) & (ml["date"] < nxt)].dropna(subset=features + ["
y_spike"])
65         if not seg.empty:
66             p = m.predict(seg[features].to_numpy("float64"))
67             oos_pred.extend(p.tolist())
68             oos_true.extend(seg["y_spike"].tolist())
69     oos_corr = float(np.corrcoef(oos_pred, oos_true)[0, 1]) if len(oos_pred) > 10 else
np.nan
70     return yhat_reb, oos_corr
71
72

```



```

73 def _high_veto(vals_at_reb: dict, q=VETO_Q, warmup=MIN_TRAIN_DAYS // 21) -> set:
74     """Veto rebalances whose value is above the expanding q-quantile of past values."""
75     "
76     dates = sorted(vals_at_reb)
77     veto, hist = set(), []
78     for d in dates:
79         v = vals_at_reb[d]
80         if len(hist) >= warmup and np.isfinite(v) and v > np.quantile(hist, q):
81             veto.add(pd.Timestamp(d))
82         if np.isfinite(v):
83             hist.append(v)
84     return veto
85
86 def _stats(q, d):
87     r, dr = q["ret_q"].astype(float), d["ret"].astype(float)
88     cum = (1 + dr).cumprod()
89     return dict(trades=int(q["traded"].sum()), n=len(q), sharpe=r.mean() / r.std() * 2,
90
91                 skew=float(r.skew()), maxDD=float((cum / cum.cummax() - 1).min()),
92                 cumul=float(cum.iloc[-1]), worst=float(r.min()))
93
94 def run_levers(processed_dir="data/processed"):
95     """Backtest lever 1 (spike) and lever 3 (regime-only) against v1_rmt, and write
96     the outputs."""
97     ml = pd.read_parquet(os.path.join(processed_dir, "ml_dataset.parquet"))
98     signal_rmt = pd.read_parquet(os.path.join(processed_dir, "signal_rmt.parquet"))
99     weights = pd.read_parquet(os.path.join(processed_dir, "weights.parquet"))
100     ml["date"] = pd.to_datetime(ml["date"])
101     signal_rmt["date"] = pd.to_datetime(signal_rmt["date"])
102     weights["rebalance_date"] = pd.to_datetime(weights["rebalance_date"])
103     rebals = sorted(weights["rebalance_date"].unique())
104     base = {kk: pd.read_parquet(os.path.join(processed_dir, f"{kk}.parquet"))
105             for kk in ("surface", "spots", "rates", "weights")}
106     v1rmt_thr = exante_quantile_threshold(signal_rmt, rebals, q=0.5, warmup=12)
107
108     # lever 3: regime probability at each rebalance
109     reg_at = {pd.Timestamp(r): float(ml.loc[ml["date"] == pd.Timestamp(r), "f_regime"].
110                                     iloc[0])
111               for r in rebals if (ml["date"] == pd.Timestamp(r)).any()
112               and np.isfinite(ml.loc[ml["date"] == pd.Timestamp(r), "f_regime"].iloc
113                                   [0]))}
114     veto_reg = _high_veto(reg_at)
115
116     # lever 1: daily spike predictor
117     yhat_reb, oos_corr = predict_spike_daily(processed_dir)
118     veto_spike = _high_veto(yhat_reb)
119
120     out = {"oos_corr_spike": oos_corr, "n_veto_regime": len(veto_reg),
121           "n_veto_spike": len(veto_spike), "arms": {}}

```

```

119 for name, veto in [("regonly", veto_reg), ("spike", veto_spike)]:
120     thr = v1rmt_thr.copy()
121     for d in veto:
122         thr[pd.Timestamp(d)] = 1e9
123     for tag, kw in [("gross", {}), ("net", dict(costs={}))]:
124         bt = run_backtest(data={**{k: v.copy() for k, v in base.items()},
125                                "signal": signal_rmt.copy()}, threshold=thr, **kw)
126         bt["quarterly"].to_parquet(
127             os.path.join(processed_dir, f"backtest_{name}_{tag}_quarterly.parquet"
128         ), index=False)
129         bt["daily"].to_parquet(
130             os.path.join(processed_dir, f"backtest_{name}_{tag}_daily.parquet"),
131             index=False)
132         out["arms"].setdefault(name, {})[tag] = _stats(bt["quarterly"], bt["daily"]
133     ])
134     json.dump(out, open(os.path.join(processed_dir, "ml_levers_summary.json"), "w"),
135                 indent=2)
136     return out

```

B.7 Shared utilities (src/dispersion/utils/)

Listing 17: src/dispersion/utils/greeks.py

```

1  """
2  Black-Scholes / Black-76 prices and greeks, with per-contract <-> relative
3  conversions. sigma is annualised IV in decimals, T in years, r and q continuous.
4
5  Relative greeks are per dollar invested (greek / price) -- that's the ratio the
6  vega hedge uses; mixing per-contract greeks with wealth weights gives a wrong hedge.
7
8  Usage:
9      from dispersion.utils.greeks import bs_greeks, relative_greeks, implied_forward
10 """
11 import numpy as np
12 from scipy.optimize import brentq
13 from scipy.stats import norm
14
15 GREEK_KEYS = ("price", "delta", "gamma", "vega", "theta", "volga", "vanna")
16
17
18 # ----- #
19 # Pricing
20 # ----- #
21 def _d1_d2(S, K, sigma, T, r, q):
22     st = sigma * np.sqrt(T)
23     d1 = (np.log(S / K) + (r - q + 0.5 * sigma**2) * T) / st
24     return d1, d1 - st
25
26
27 def bs_price(S, K, sigma, T, r, q, cp: str) -> float:
28     """European option price, spot form with continuous dividend yield q."""

```

```

29 d1, d2 = _d1_d2(S, K, sigma, T, r, q)
30 if cp == "C":
31     return S * np.exp(-q * T) * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
32 if cp == "P":
33     return K * np.exp(-r * T) * norm.cdf(-d2) - S * np.exp(-q * T) * norm.cdf(-d1)
34 raise ValueError(f"cp must be 'C' or 'P', got {cp!r}")
35
36
37 def black76_price(F, K, sigma, T, r, cp: str) -> float:
38     """European option price, forward form (Black-76): discounts the forward payoff."""
39     # same as the spot form with S = F*exp(-(r-q)*T)
40     st = sigma * np.sqrt(T)
41     d1 = (np.log(F / K) + 0.5 * sigma**2 * T) / st
42     d2 = d1 - st
43     df = np.exp(-r * T)
44     if cp == "C":
45         return df * (F * norm.cdf(d1) - K * norm.cdf(d2))
46     if cp == "P":
47         return df * (K * norm.cdf(-d2) - F * norm.cdf(-d1))
48     raise ValueError(f"cp must be 'C' or 'P', got {cp!r}")
49
50
51 # ----- #
52 # Greeks (per contract)
53 # ----- #
54 def bs_greeks(S, K, sigma, T, r, q, cp: str) -> dict:
55     """
56     Price plus first- and second-order greeks per contract, spot form.
57     Returns dict(price, delta, gamma, vega, theta, volga, vanna).
58
59     volga and vanna carry the book's convexity risk:
60     volga = d(vega)/d(sigma) = vega * d1 * d2 / sigma
61     vanna = d(vega)/d(S) = d(delta)/d(sigma)
62     Both vanish at the ATM-forward and grow as a straddle drifts off ATM.
63     """
64     d1, d2 = _d1_d2(S, K, sigma, T, r, q)
65     eq, er = np.exp(-q * T), np.exp(-r * T)
66     pdf1 = norm.pdf(d1)
67
68     gamma = eq * pdf1 / (S * sigma * np.sqrt(T))
69     vega = S * eq * pdf1 * np.sqrt(T)
70     volga = vega * d1 * d2 / sigma
71     vanna = -eq * pdf1 * d2 / sigma
72     common_theta = -S * eq * pdf1 * sigma / (2.0 * np.sqrt(T))
73
74     if cp == "C":
75         price = S * eq * norm.cdf(d1) - K * er * norm.cdf(d2)
76         delta = eq * norm.cdf(d1)
77         theta = common_theta - r * K * er * norm.cdf(d2) + q * S * eq * norm.cdf(d1)
78     elif cp == "P":

```

```

79     price = K * er * norm.cdf(-d2) - S * eq * norm.cdf(-d1)
80     delta = -eq * norm.cdf(-d1)
81     theta = common_theta + r * K * er * norm.cdf(-d2) - q * S * eq * norm.cdf(-d1)
82 else:
83     raise ValueError(f"cp must be 'C' or 'P', got {cp!r}")
84
85     return {"price": float(price), "delta": float(delta), "gamma": float(gamma),
86            "vega": float(vega), "theta": float(theta),
87            "volga": float(volga), "vanna": float(vanna)}
88
89
90 def straddle_greeks(S, K_call, K_put, sigma_call, sigma_put, T, r, q) -> dict:
91     """Greeks of a standardised straddle = call + put legs (strikes may differ
92     slightly)."""
93     c = bs_greeks(S, K_call, sigma_call, T, r, q, "C")
94     p = bs_greeks(S, K_put, sigma_put, T, r, q, "P")
95     return {k: c[k] + p[k] for k in GREEK_KEYS}
96
97 # ----- #
98 # Per-contract <-> relative (per dollar invested)
99 # ----- #
100 def relative_greeks(greeks: dict) -> dict:
101     """
102     Per-contract greeks -> per-dollar-invested greeks (greek / price).
103
104     These are what go into the wealth-weighted hedge ratio
105     y_i = vega_rel_index / vega_rel_component; per-contract vegas with wealth
106     weights give the wrong hedge.
107     """
108     price = greeks["price"]
109     if price <= 0:
110         raise ValueError("relative greeks undefined for non-positive price")
111     out = {k: greeks[k] / price for k in GREEK_KEYS if k != "price"}
112     out["price"] = price
113     return out
114
115
116 # ----- #
117 # Implied quantities
118 # ----- #
119 def implied_forward(call_price, put_price, K, T, r) -> float:
120     """Model-free forward from put-call parity at a common strike:  $F = K + e^{rT}(C - P)$ ."""
121     return float(K + np.exp(r * T) * (call_price - put_price))
122
123
124 def implied_vol(price, S, K, T, r, q, cp: str, lo=1e-4, hi=5.0) -> float:
125     """Invert bs_price for sigma (brentq)."""
126     return float(brentq(lambda s: bs_price(S, K, s, T, r, q, cp) - price, lo, hi))

```

B.8 Tests (tests/)

Listing 18: tests/test_greeks.py

```
1  """
2  Tests for dispersion.utils.greeks: closed-form values and identities, greeks vs
3  finite differences, the relative-vs-per-contract vega hedge, and reproducing real
4  SPX premiums.
5  """
6  import numpy as np
7  import pytest
8  from scipy.stats import norm
9
10 from dispersion.utils.greeks import (
11     bs_greeks,
12     bs_price,
13     black76_price,
14     implied_forward,
15     implied_vol,
16     relative_greeks,
17     straddle_greeks,
18 )
19
20 # non-trivial base case: ITM-ish call, with dividends and rates
21 BASE = dict(S=100.0, K=95.0, sigma=0.25, T=0.25, r=0.04, q=0.015)
22
23
24 # ----- #
25 # 1. Closed form & identities
26 # ----- #
27 def test_atm_forward_closed_form():
28     # BS price against its exact closed form when r = q = 0 and K = S
29     S = K = 100.0
30     sigma, T = 0.20, 91 / 365
31     d = 0.5 * sigma * np.sqrt(T)
32     expected_call = S * (norm.cdf(d) - norm.cdf(-d))
33     assert bs_price(S, K, sigma, T, 0.0, 0.0, "C") == pytest.approx(expected_call, rel
34                               =1e-12)
35     # ATM-forward, so call and put match
36     assert bs_price(S, K, sigma, T, 0.0, 0.0, "P") == pytest.approx(expected_call, rel
37                               =1e-12)
38
39 def test_put_call_parity_and_symmetries():
40     rng = np.random.default_rng(42)
41     for _ in range(200):
42         S = rng.uniform(20, 500)
43         K = S * rng.uniform(0.7, 1.3)
44         sigma = rng.uniform(0.05, 0.9)
45         T = rng.uniform(0.02, 2.0)
46         r = rng.uniform(0.0, 0.08)
```

```

46     q = rng.uniform(0.0, 0.05)
47     c = bs_greeks(S, K, sigma, T, r, q, "C")
48     p = bs_greeks(S, K, sigma, T, r, q, "P")
49     # put-call parity:  $C - P = S e^{-qT} - K e^{-rT}$ 
50     assert c["price"] - p["price"] == pytest.approx(
51         S * np.exp(-q * T) - K * np.exp(-r * T), abs=1e-9
52     )
53     #  $\Delta_C - \Delta_P = e^{-qT}$ ; gamma and vega equal across the two legs
54     assert c["delta"] - p["delta"] == pytest.approx(np.exp(-q * T), abs=1e-12)
55     assert c["gamma"] == pytest.approx(p["gamma"], rel=1e-12)
56     assert c["vega"] == pytest.approx(p["vega"], rel=1e-12)
57
58
59 def test_black76_equals_spot_form():
60     S, K, sigma, T, r, q = BASE.values()
61     F = S * np.exp((r - q) * T)
62     for cp in ("C", "P"):
63         assert black76_price(F, K, sigma, T, r, cp) == pytest.approx(
64             bs_price(S, K, sigma, T, r, q, cp), rel=1e-12
65         )
66
67
68 def test_implied_forward_parity():
69     S, K, sigma, T, r, q = BASE.values()
70     c = bs_price(S, K, sigma, T, r, q, "C")
71     p = bs_price(S, K, sigma, T, r, q, "P")
72     assert implied_forward(c, p, K, T, r) == pytest.approx(S * np.exp((r - q) * T),
73 rel=1e-12)
74
75
76 def test_implied_vol_round_trip():
77     S, K, sigma, T, r, q = BASE.values()
78     for cp in ("C", "P"):
79         price = bs_price(S, K, sigma, T, r, q, cp)
80         assert implied_vol(price, S, K, T, r, q, cp) == pytest.approx(sigma, abs=1e
81 -10)
82
83
84 def test_straddle_is_sum_of_legs():
85     S, K, sigma, T, r, q = BASE.values()
86     st = straddle_greeks(S, K_call=K, K_put=K + 1, sigma_call=sigma, sigma_put=sigma +
87 0.01,
88 T=T, r=r, q=q)
89     c = bs_greeks(S, K, sigma, T, r, q, "C")
90     p = bs_greeks(S, K + 1, sigma + 0.01, T, r, q, "P")
91     for k in ("price", "delta", "gamma", "vega", "theta"):
92         assert st[k] == pytest.approx(c[k] + p[k], rel=1e-12)
93
94 # ----- #
95 # 2. Greeks vs bump-and-reprice

```

```

94 # ----- #
95 @pytest.mark.parametrize("cp", ["C", "P"])
96 def test_greeks_match_finite_differences(cp):
97     S, K, sigma, T, r, q = BASE.values()
98     g = bs_greeks(S, K, sigma, T, r, q, cp)
99     hs, h = 1e-3, 1e-5 # bigger S bump for the second derivative (cancellation noise
    ~1/h^2)
100
101     delta_fd = (bs_price(S + hs, K, sigma, T, r, q, cp) - bs_price(S - hs, K, sigma, T,
    r, q, cp)) / (2 * hs)
102     gamma_fd = (bs_price(S + hs, K, sigma, T, r, q, cp) - 2 * bs_price(S, K, sigma, T,
    r, q, cp)
103                 + bs_price(S - hs, K, sigma, T, r, q, cp)) / hs**2
104     vega_fd = (bs_price(S, K, sigma + h, T, r, q, cp) - bs_price(S, K, sigma - h, T, r,
    q, cp)) / (2 * h)
105     # theta is the derivative w.r.t. time passing, i.e. -dP/dT
106     theta_fd = -(bs_price(S, K, sigma, T + h, r, q, cp) - bs_price(S, K, sigma, T - h,
    r, q, cp)) / (2 * h)
107
108     assert g["delta"] == pytest.approx(delta_fd, rel=1e-7)
109     assert g["gamma"] == pytest.approx(gamma_fd, rel=1e-5)
110     assert g["vega"] == pytest.approx(vega_fd, rel=1e-7)
111     assert g["theta"] == pytest.approx(theta_fd, rel=1e-6)
112
113     # second order: volga = d(vega)/d(sigma), vanna = d(vega)/d(S)
114     volga_fd = (bs_greeks(S, K, sigma + h, T, r, q, cp)["vega"]
115                 - bs_greeks(S, K, sigma - h, T, r, q, cp)["vega"]) / (2 * h)
116     vanna_fd = (bs_greeks(S + hs, K, sigma, T, r, q, cp)["vega"]
117                 - bs_greeks(S - hs, K, sigma, T, r, q, cp)["vega"]) / (2 * hs)
118     assert g["volga"] == pytest.approx(volga_fd, rel=1e-5)
119     assert g["vanna"] == pytest.approx(vanna_fd, rel=1e-5)
120
121
122 def test_second_order_greeks_vanish_at_d2_zero_strike():
123     # volga ( $\propto d1 \cdot d2$ ) and vanna ( $\propto d2$ ) are both zero at the strike where  $d2 = 0$ ,
124     # and grow as a straddle ages away from it -- that drift is the convexity risk.
125     S, sigma, T, r, q = 100.0, 0.25, 0.5, 0.03, 0.01
126     K0 = S * np.exp((r - q - 0.5 * sigma**2) * T) # d2 = 0
127     g0 = bs_greeks(S, K0, sigma, T, r, q, "C")
128     assert abs(g0["vanna"]) < 1e-9 and abs(g0["volga"]) < 1e-9
129     # spot drifted down 15%: both are clearly non-zero
130     g1 = bs_greeks(S * 0.85, K0, sigma, T, r, q, "C")
131     assert abs(g1["vanna"]) > 1e-3 and abs(g1["volga"]) > 1e-2
132
133
134 # ----- #
135 # 3. Relative vs per-contract hedge ratios
136 # ----- #
137 def test_relative_vega_hedge_is_wealth_neutral_and_per_contract_is_not():
138     T, r, q = 0.25, 0.04, 0.0
139     # expensive index-like straddle vs cheap single-name-like straddle

```

```

140 idx = straddle_greeks(S=5000.0, K_call=5000.0, K_put=5000.0,
141                       sigma_call=0.12, sigma_put=0.12, T=T, r=r, q=q)
142 comp = straddle_greeks(S=50.0, K_call=50.0, K_put=50.0,
143                       sigma_call=0.30, sigma_put=0.30, T=T, r=r, q=q)
144 idx_rel, comp_rel = relative_greeks(idx), relative_greeks(comp)
145
146 # hedge ratio from the relative vegas
147 y = idx_rel["vega"] / comp_rel["vega"]
148 # short $1 of index, long $y of component -> zero wealth-vega
149 wealth_vega = -1.0 * idx_rel["vega"] + y * comp_rel["vega"]
150 assert wealth_vega == pytest.approx(0.0, abs=1e-12)
151
152 # using the per-contract vega ratio instead leaves a large residual
153 y_trap = idx["vega"] / comp["vega"]
154 wealth_vega_trap = -1.0 * idx_rel["vega"] + y_trap * comp_rel["vega"]
155 assert abs(wealth_vega_trap) > 0.1 * idx_rel["vega"] # off by a lot, not rounding
156
157 # the two ratios differ by exactly the price ratio
158 assert y_trap / y == pytest.approx(idx["price"] / comp["price"], rel=1e-12)
159
160
161 # ----- #
162 # 4. Real data: SPX standardised 91d  $\pm 50\delta$ , 2024-06-28
163 # ----- #
164 def test_reproduces_spx_impl_premium_2024():
165     S, T, r = 5460.48, 91 / 365, 0.0550 # secprd close, zerocd 91d
166     q = 0.0104 # implied dividend yield, same for both legs
167     call = bs_price(S, 5530.623, 0.121323, T, r, q, "C")
168     put = bs_price(S, 5531.409, 0.113183, T, r, q, "P")
169     assert call == pytest.approx(127.2951, rel=5e-3)
170     assert put == pytest.approx(127.7991, rel=5e-3)

```

Listing 19: tests/test_marking.py

```

1 """
2 Tests for dispersion.backtest.marking: the mark-to-surface rules.
3 """
4 import numpy as np
5 import pandas as pd
6 import pytest
7
8 from dispersion.backtest.marking import RateCurve, adjust_strike, interp_sigma
9
10
11 # ----- #
12 # interp_sigma -- total-variance interpolation + frozen short end
13 # ----- #
14 def test_exact_at_pillars():
15     s30, s60, s91 = 0.30, 0.25, 0.22
16     assert interp_sigma(s30, s60, s91, 30) == pytest.approx(s30, rel=1e-12)
17     assert interp_sigma(s30, s60, s91, 60) == pytest.approx(s60, rel=1e-12)

```



```

18     assert interp_sigma(s30, s60, s91, 91) == pytest.approx(s91, rel=1e-12)
19
20
21 def test_linear_in_total_variance_between_pillars():
22     s30, s60, s91 = 0.30, 0.25, 0.22
23     # tau = 45 sits halfway between the 30 and 60 pillars in total variance
24     w30, w60 = s30**2 * 30, s60**2 * 60
25     expected = np.sqrt(((w30 + w60) / 2) / 45)
26     assert interp_sigma(s30, s60, s91, 45) == pytest.approx(expected, rel=1e-12)
27     # tau = 75.5, midpoint of [60, 91]
28     w91 = s91**2 * 91
29     expected = np.sqrt(((w60 + w91) / 2) / 75.5)
30     assert interp_sigma(s30, s60, s91, 75.5) == pytest.approx(expected, rel=1e-12)
31
32
33 def test_frozen_below_30d():
34     s30, s60, s91 = 0.30, 0.25, 0.22
35     for tau in (29.9, 15, 5, 1):
36         assert interp_sigma(s30, s60, s91, tau) == pytest.approx(s30, rel=1e-12)
37
38
39 def test_vectorised_and_nan_propagation():
40     s30 = np.array([0.30, np.nan, 0.40])
41     s60 = np.array([0.25, 0.25, 0.35])
42     s91 = np.array([0.22, 0.22, 0.33])
43     out = interp_sigma(s30, s60, s91, np.array([45.0, 45.0, 20.0]))
44     assert out.shape == (3,)
45     assert np.isnan(out[1]) # a NaN pillar propagates to the output
46     assert out[2] == pytest.approx(0.40) # short end frozen element by element
47
48
49 # ----- #
50 # RateCurve -- maturity interpolation + bounded date-ffill
51 # ----- #
52 def _toy_rates():
53     return pd.DataFrame({
54         "date": pd.to_datetime(["2020-01-02"] * 3 + ["2020-01-06"] * 3),
55         "days": [30, 91, 182] * 2,
56         "rate": [1.0, 2.0, 3.0, 1.5, 2.5, 3.5], # percent
57     })
58
59
60 def test_rate_exact_and_interpolated():
61     rc = RateCurve(_toy_rates())
62     assert rc.rate("2020-01-02", 91) == pytest.approx(0.02, abs=1e-12)
63     # midpoint of [30, 91]
64     assert rc.rate("2020-01-02", 60.5) == pytest.approx(0.015, abs=1e-12)
65     # off the ends of the grid -> clamped
66     assert rc.rate("2020-01-02", 10) == pytest.approx(0.01, abs=1e-12)
67     assert rc.rate("2020-01-02", 400) == pytest.approx(0.03, abs=1e-12)
68

```

```

69
70 def test_rate_bounded_ffill_and_loud_failure():
71     rc = RateCurve(_toy_rates(), max_stale_days=7)
72     # 01-03 has no curve, so reuse 01-02 (1 day stale, within bound)
73     assert rc.rate("2020-01-03", 91) == pytest.approx(0.02, abs=1e-12)
74     # past the staleness bound -> raise rather than use a stale curve
75     with pytest.raises(KeyError):
76         rc.rate("2020-02-01", 91)
77     # before the first curve -> raise
78     with pytest.raises(KeyError):
79         rc.rate("2019-12-31", 91)
80
81
82 # ----- #
83 # adjust_strike -- split handling via cfadj
84 # ----- #
85 def test_adjust_strike_split():
86     # 2:1 split: cfadj goes 1 -> 2, price halves, so the strike halves too
87     assert adjust_strike(100.0, 1.0, 2.0) == pytest.approx(50.0)
88     # no split leaves the strike alone
89     assert adjust_strike(100.0, 1.0, 1.0) == pytest.approx(100.0)
90     # once both are rebased, |S - K| is unchanged by the split
91     S_pre, K_pre = 120.0, 100.0
92     S_post, cf_e, cf_t = 60.0, 1.0, 2.0
93     K_post = adjust_strike(K_pre, cf_e, cf_t)
94     qty_scale = cf_t / cf_e
95     assert qty_scale * abs(S_post - K_post) == pytest.approx(abs(S_pre - K_pre))

```

Listing 20: tests/test_engine.py

```

1  """
2  Tests for dispersion.backtest.engine on a synthetic quarter with a closed-form
3  answer. With r = 0, flat sigmas, constant spots, ATM strikes and q = 0, the
4  settlement payoffs are 0 and daily hedging is cash-neutral, so the quarterly
5  return is exactly 1 - sum(y_i). Sizing identities are checked the same way.
6  """
7  import numpy as np
8  import pandas as pd
9  import pytest
10
11  from dispersion.backtest.engine import (SPX_SECID, exante_quantile_threshold,
12                                         implied_q, run_backtest)
13  from dispersion.utils.greeks import bs_greeks, bs_price
14
15  REB = pd.Timestamp("2020-03-31")
16  NXT = pd.Timestamp("2020-06-30")          # 91 calendar days
17  TO = (NXT - REB).days / 365.0
18  NAMES = {1: dict(S=100.0, w=0.6), 2: dict(S=50.0, w=0.4)}
19  SPX_S = 1000.0
20
21

```

```

22 def _make_data(sig_by_secid):
23     dates = pd.DatetimeIndex([REB, "2020-04-30", "2020-05-29", "2020-06-29"])
24     surf, spot, rate, wgt = [], [], [], []
25     all_ids = {SPX_SECID: SPX_S, **{s: NAMES[s]["S"] for s in NAMES}}
26
27     for secid, S in all_ids.items():
28         sig = sig_by_secid[secid]
29         prem = {cp: bs_price(S, S, sig, T0, 0.0, 0.0, cp) for cp in ("C", "P")}
30         for d in dates:
31             for days in (30, 60, 91):
32                 for cp in ("C", "P"):
33                     surf.append((REB, d, secid, days, cp, sig, prem[cp], S))
34                     spot.append((REB, d, secid, S, 1.0))
35     # settlement rows live in the next quarter's partition (date == NXT)
36     for secid, S in all_ids.items():
37         spot.append((NXT, NXT, secid, S, 1.0))
38
39     for d in list(dates) + [NXT]:
40         for days in (10, 30, 60, 91, 122):
41             rate.append((d, days, 0.0))
42
43     for rnk, (secid, info) in enumerate(NAMES.items(), start=1):
44         wgt.append((REB, secid, info["w"], rnk))
45         wgt.append((NXT, secid, info["w"], rnk)) # gives NXT a rebalance so
46     settlement fires
47
48     return {
49         "surface": pd.DataFrame(surf, columns=["rebalance_date", "date", "secid",
50             "days", "cp_flag", "iv", "premium",
51             "strike"]),
52         "spots": pd.DataFrame(spot, columns=["rebalance_date", "date", "secid",
53             "close", "cfadj"]),
54         "rates": pd.DataFrame(rate, columns=["date", "days", "rate"]),
55         "weights": pd.DataFrame(wgt, columns=["rebalance_date", "secid", "weight", "
56     rnk"]),
57         "signal": pd.DataFrame({"date": [REB], "signal": [0.10]}),
58     }
59
60 def test_implied_q_roundtrip():
61     S, k_c, k_p, sc, sp_, T, r, q_true = 100.0, 101.0, 99.0, 0.25, 0.27, 0.25, 0.03,
62     0.017
63     c = bs_price(S, k_c, sc, T, r, q_true, "C")
64     p = bs_price(S, k_p, sp_, T, r, q_true, "P")
65     assert implied_q(S, k_c, k_p, sc, sp_, T, r, c, p) == pytest.approx(q_true, abs=1e
66     -8)
67
68 def test_flat_quarter_closed_form():
69     # same sigma everywhere -> nu*sigma equal across names -> lam = 1, y_sum = 1
70     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})

```

```

69     res = run_backtest(data=data, min_names=2)
70     q = res["quarterly"].iloc[0]
71
72     assert q["traded"] and q["n_names"] == 2
73     assert q["lam"] == pytest.approx(1.0, rel=1e-10)
74     assert q["y_sum"] == pytest.approx(1.0, rel=1e-10)
75     # zero settlement, cash-neutral hedge, r = 0: quarterly return is 1 - y_sum
76     assert q["ret_q"] == pytest.approx(1.0 - q["y_sum"], abs=1e-12)
77     # compounded daily returns match the quarterly return
78     d = res["daily"]
79     assert (1 + d["ret"]).prod() - 1 == pytest.approx(q["ret_q"], abs=1e-12)
80     assert q["avg_frozen_days"] == 0 and q["n_frozen_settle"] == 0
81
82
83 def test_sizing_identity_heterogeneous_vols():
84     sig = {SPX_SECID: 0.18, 1: 0.35, 2: 0.22}
85     data = _make_data(sig)
86     res = run_backtest(data=data, min_names=2)
87     q = res["quarterly"].iloc[0]
88
89     # closed-form lam: nu = straddle vega / straddle price at entry
90     def nu(S, s):
91         g_c = bs_greeks(S, S, s, T0, 0.0, 0.0, "C")
92         g_p = bs_greeks(S, S, s, T0, 0.0, 0.0, "P")
93         return (g_c["vega"] + g_p["vega"]) / (g_c["price"] + g_p["price"])
94
95     nu_i = nu(SPX_S, sig[SPX_SECID])
96     denom = sum(NAMES[s]["w"] * nu(NAMES[s]["S"], sig[s]) * sig[s] for s in NAMES)
97     lam_expected = nu_i * sig[SPX_SECID] / denom
98     assert q["lam"] == pytest.approx(lam_expected, rel=1e-9)
99
100     # wealth-vega neutrality under a proportional shock holds by construction
101     lhs = sum(q["lam"] * NAMES[s]["w"] * nu(NAMES[s]["S"], sig[s]) * sig[s] for s in
102 NAMES)
103     assert lhs == pytest.approx(nu_i * sig[SPX_SECID], rel=1e-9)
104
105 def test_settlement_intrinsic_payoff():
106     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
107     # move name 1's settlement spot so its straddle pays |120 - 100| = 20 per unit
108     sp = data["spots"]
109     sp.loc[(sp["rebalance_date"] == NXT) & (sp["secid"] == 1), "close"] = 120.0
110     res = run_backtest(data=data, min_names=2)
111     q = res["quarterly"].iloc[0]
112
113     # flat-case return (1 - y_sum) plus name 1's settlement payoff
114     price_1 = bs_price(100.0, 100.0, 0.20, T0, 0.0, 0.0, "C") + \
115         bs_price(100.0, 100.0, 0.20, T0, 0.0, 0.0, "P")
116     y_1 = q["lam"] * NAMES[1]["w"]
117     expected = (1.0 - q["y_sum"]) + y_1 / price_1 * 20.0
118     assert q["ret_q"] == pytest.approx(expected, abs=1e-12)

```

```

119
120
121 def test_parsimonious_leg_keeps_vega_neutrality():
122     # n_leg=1 keeps only the top-weight name but rescales it, so wealth-vega
123     # neutrality (y_sum ~ 1 in the flat case) and ret_q = 1 - y_sum still hold.
124     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
125     res = run_backtest(data=data, min_names=2, n_leg=1)
126     q = res["quarterly"].iloc[0]
127     assert q["n_names"] == 1 # only the top-weight component
128     assert q["y_sum"] == pytest.approx(1.0, rel=1e-10) # still wealth-vega neutral
129     assert q["ret_q"] == pytest.approx(1.0 - q["y_sum"], abs=1e-12)
130
131
132 def test_v1_gate_skips_quarter():
133     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
134     res = run_backtest(data=data, min_names=2, threshold=0.50) # signal = 0.10 < 0.50
135     q = res["quarterly"].iloc[0]
136     assert not q["traded"]
137     assert q["ret_q"] == pytest.approx(0.0, abs=1e-12) # nothing traded, r =
138     0 -> flat
139
140 def test_costs_closed_form():
141     from dispersion.backtest.engine import parametric_spread
142     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
143     res_g = run_backtest(data=data, min_names=2)
144     res_n = run_backtest(data=data, min_names=2, costs={"hedge_bps": 0.0})
145     qg, qn = res_g["quarterly"].iloc[0], res_n["quarterly"].iloc[0]
146     # entry cost = half-spread on the index leg (100% of W) plus each component leg
147     expected = 0.5 * parametric_spread(REB.year, "spx") \
148         + 0.5 * parametric_spread(REB.year, "large") * qg["y_sum"] # rnk 1-2 map to "
149     large"
150     assert qn["cost_entry"] == pytest.approx(expected, rel=1e-10)
151     assert qn["cost_hedge"] == 0.0
152     assert qn["ret_q"] == pytest.approx(qg["ret_q"] - expected, abs=1e-12)
153
154 def test_exante_quantile_threshold_values():
155     sig = pd.DataFrame({"date": pd.date_range("2020-01-01", periods=100, freq="D"),
156         "signal": np.linspace(0.0, 1.0, 100)})
157     rebs = [pd.Timestamp("2020-02-01"), pd.Timestamp("2020-04-01")]
158     thr = exante_quantile_threshold(sig, rebs, q=0.5, warmup=1)
159     assert np.isnan(thr.iloc[0]) # still warming up
160     expected = sig.set_index("date")["signal"].loc["2020-04-01"].median()
161     assert thr.iloc[1] == pytest.approx(expected, rel=1e-12) # median of history so
162     far
163
164 def test_v1_series_gate_warmup_trades():
165     data = _make_data({SPX_SECID: 0.20, 1: 0.20, 2: 0.20})
166     # NaN threshold at REB (warm-up) trades like v0

```

```

167 thr = pd.Series({REB: np.nan})
168 res = run_backtest(data=data, min_names=2, threshold=thr)
169 assert bool(res["quarterly"].iloc[0]["traded"])
170 # threshold above the signal (0.10) skips
171 thr = pd.Series({REB: 0.50})
172 res = run_backtest(data=data, min_names=2, threshold=thr)
173 assert not bool(res["quarterly"].iloc[0]["traded"])

```

Listing 21: tests/test_rmt.py

```

1 """
2 Tests for dispersion.rmt.cleaning: the Marchenko-Pastur + Laloux cleaning.
3 """
4 import numpy as np
5 import pandas as pd
6 import pytest
7
8 from dispersion.rmt.cleaning import (corr_window, devolatilise, laloux_clip,
9                                     spectral_features)
10
11
12 def _corr_df(A):
13     n = A.shape[0]
14     idx = list(range(n))
15     return pd.DataFrame(A, index=idx, columns=idx)
16
17
18 # ----- #
19 # pure noise in -> filter finds nothing, comes out near-identity
20 # ----- #
21 def test_iid_noise_cleans_to_near_identity():
22     rng = np.random.default_rng(7)
23     X = rng.standard_normal((252, 100))
24     C = _corr_df(np.corrcoef(X, rowvar=False))
25     C2, d = laloux_clip(C, t_win=252)
26
27     off_raw = np.abs(C.values[np.triu_indices(100, 1)]).mean()
28     off_clean = np.abs(C2.values[np.triu_indices(100, 1)]).mean()
29     assert d["k_signal"] == 0 # no signal invented from noise
30     assert off_clean < 0.05 * off_raw # off-diagonals collapse to ~
31     identity
32     assert d["trace_prenorm"] == pytest.approx(100.0, rel=1e-10)
33
34 def test_diag_psd_trace_on_correlated_data():
35     rng = np.random.default_rng(11)
36     # one-factor returns: beta * market + idiosyncratic
37     mkt = rng.standard_normal(252)
38     X = 0.6 * mkt[:, None] + rng.standard_normal((252, 80))
39     C = _corr_df(np.corrcoef(X, rowvar=False))
40     C2, d = laloux_clip(C, t_win=252)

```

```

41
42     assert np.allclose(np.diag(C2.values), 1.0, atol=1e-12)           # diagonal stays
43     1
44     assert np.linalg.eigvalsh(C2.values).min() > -1e-9              # PSD
45     assert d["trace_prenorm"] == pytest.approx(80.0, rel=1e-10)      # trace preserved
46     assert d["k_signal"] >= 1                                       # market mode
47     kept
48
49     assert d["edge_laloux"] < d["edge_naive"]                        # Laloux edge
50     below the naive one
51
52 def test_equicorrelation_structure_survives_cleaning():
53     # exact one-factor matrix C = (1-rho) I + rho J: cleaning should leave it alone
54     n, rho = 100, 0.3
55     C = _corr_df((1 - rho) * np.eye(n) + rho * np.ones((n, n)))
56     C2, d = laloux_clip(C, t_win=252)
57     assert np.abs(C2.values - C.values).max() < 1e-10
58     assert d["k_signal"] == 1                                       # one mode: the
59     market
60
61 def test_devolatilise_scale_invariance():
62     # rescaling each name in log-return space must not change the z-correlations
63     # (log1p is nonlinear in simple returns, so the rescale is done in log space)
64     rng = np.random.default_rng(3)
65     dates = pd.date_range("2020-01-01", periods=300, freq="B")
66     r1 = pd.DataFrame(rng.standard_normal((300, 4)) * 0.01, index=dates)
67     r2 = np.exp1(np.log1p(r1) * [1.0, 5.0, 0.2, 3.0]) # per-name log-scale
68     rescaling
69     c1 = devolatilise(r1).tail(200).dropna().corr().values
70     c2 = devolatilise(r2).tail(200).dropna().corr().values
71     assert np.abs(c1 - c2).max() < 1e-10                          # correlations unchanged by
72     scale
73
74 def test_corr_window_thin_panel_returns_none():
75     rng = np.random.default_rng(5)
76     dates = pd.date_range("2020-01-01", periods=300, freq="B")
77     z = pd.DataFrame(rng.standard_normal((300, 50)), index=dates)
78     assert corr_window(z, dates[-1], list(range(50)), min_names=60) is None
79
80 def test_corr_window_returns_effective_t():
81     rng = np.random.default_rng(9)
82     dates = pd.date_range("2019-01-01", periods=400, freq="B")
83     z = pd.DataFrame(rng.standard_normal((400, 70)), index=dates)
84     z.iloc[-30:-20, 0] = np.nan                                     # 10 missing days on one name
85     C, t_eff = corr_window(z, dates[-1], list(range(70)))
86     assert t_eff == 242 and C.shape == (70, 70)                   # complete-case: 252 - 10

```

```

86 def test_effective_t_kills_spurious_signal_on_noise():
87     # with T_eff = 201, the MP edge must use q_mp = N/201, not N/252
88     rng = np.random.default_rng(21)
89     X = rng.standard_normal((201, 100))
90     C = _corr_df(np.corrcoef(X, rowvar=False))
91     _, d_wrong = laloux_clip(C, t_win=252)           # nominal T (the old bug)
92     _, d_right = laloux_clip(C, t_win=201)          # effective T
93     assert d_right["k_signal"] == 0                 # nothing in pure noise
94     assert d_wrong["k_signal"] > 0                  # wrong T would invent signal
95
96
97 def test_rotation_metric():
98     from dispersion.rmt.daily import _rotation
99     idx = list(range(20))
100     v = pd.Series(np.ones(20) / np.sqrt(20), index=idx)
101     assert _rotation(v, v) == pytest.approx(0.0, abs=1e-12)    # same vector ->
102     0                                                         # sign ignored
103     w = pd.Series(np.zeros(20), index=idx); w.iloc[0] = 1.0
104     v2 = pd.Series(np.zeros(20), index=idx); v2.iloc[1] = 1.0
105     assert _rotation(v2, w) == pytest.approx(1.0, abs=1e-12)    # orthogonal -> 1
106     assert np.isnan(_rotation(v, None))                  # no prior vector
107     -> NaN
108     u = pd.Series(np.ones(5), index=list(range(100, 105)))
109     assert np.isnan(_rotation(v, u))                      # overlap under
110     10 names -> NaN
111
112 def test_spectral_features_consistency():
113     rng = np.random.default_rng(13)
114     mkt = rng.standard_normal(252)
115     X = 0.7 * mkt[:, None] + rng.standard_normal((252, 90))
116     C = _corr_df(np.corrcoef(X, rowvar=False))
117     feats, v1 = spectral_features(C, t_win=252, top=5)
118     _, d = laloux_clip(C, t_win=252)
119     assert feats["k_signal"] == d["k_signal"]
120     assert 0 < feats["lam1_share"] < 1
121     assert feats["absorption_top"] > feats["lam1_share"]    # top-5 includes
122     top-1                                                    # sign convention
123     assert v1.sum() > 0                                     # unit norm
124     assert np.isclose((v1**2).sum(), 1.0)

```

Listing 22: tests/test_regime.py

```

1 """
2 Causality tests for dispersion.ml.regime: the HMM filtered posterior must use
3 only past and present observations, never the future.
4 """
5 import numpy as np
6 import pytest
7 from hmmlearn.hmm import GaussianHMM

```



```

8
9 from dispersion.ml.regime import hmm_filtered_posterior
10
11
12 def _fit_hmm(X, seed=0):
13     m = GaussianHMM(3, covariance_type="full", random_state=seed, n_iter=50, tol=1e-3)
14     m.fit(X)
15     return m
16
17
18 def test_filtered_posterior_is_causal():
19     # perturbing the future must leave every past filtered posterior unchanged
20     rng = np.random.default_rng(1)
21     X = rng.standard_normal((400, 4))
22     m = _fit_hmm(X)
23     post = hmm_filtered_posterior(m, X)
24
25     t0 = 250
26     X2 = X.copy()
27     X2[t0 + 1:] += 5.0 * rng.standard_normal((len(X) - t0 - 1, 4)) # scramble the
28     # future
29     post2 = hmm_filtered_posterior(m, X2)
30
31     assert np.allclose(post[:t0 + 1], post2[:t0 + 1], atol=1e-12) # past unchanged
32     assert not np.allclose(post[t0 + 1:], post2[t0 + 1:]) # future did
33     # change
34
35 def test_filtered_rows_are_probabilities():
36     rng = np.random.default_rng(2)
37     X = rng.standard_normal((300, 4))
38     post = hmm_filtered_posterior(_fit_hmm(X), X)
39     assert np.allclose(post.sum(axis=1), 1.0, atol=1e-10)
40     assert (post >= 0).all() and (post <= 1).all()
41
42 def test_filtered_equals_truncated_smoothed_at_last_step():
43     # filtered posterior at t equals the smoothed posterior of the sequence cut off
44     # at t (no future left to smooth over) -- cross-checked against hmmlearn
45     rng = np.random.default_rng(3)
46     X = rng.standard_normal((120, 4))
47     m = _fit_hmm(X)
48     filt = hmm_filtered_posterior(m, X)
49     for t in (30, 60, 119):
50         smoothed_trunc = m.predict_proba(X[:t + 1])[-1] # last row, no future
51         assert np.allclose(filt[t], smoothed_trunc, atol=1e-8)

```